Deep Hedging in Low-Data Environments using Path Signatures

Author: Nick Bossi

Supervisor: Associate Professor Jonathan Shock

Co-Supervisor: Yuri Robbertze

UCT Department of Mathematics and Applied Mathematics

27 September 2024

Contents

List of Figures

List of Tables

1 Abstract

Historically, many approaches of hedging a financial derivative have been explored. This paper attempts to answer whether or not Deep Neural Networks (DNNs) can provide robust tools for the hedging of a European Best-of option. As we are using deep learning techniques, we face the issue of not having sufficient real-world data in order to effectively train our model. We thus employ the use of a Conditional Variational Autoencoder as a tool to learn the underlying stock's distributions so that we can simulate a range of psuedo-real-world future paths. The advantage of this is two-fold, as it provides realistic training data for our deep hedger, as well as giving us a theoretical means to test the robustness of our hedger under many future market conditions. Instead of directly using time-series stock data, we make use of a feature mapping for time-series data known as a path signature, which has been shown to effectively capture key information of the joint-distributions of temporal data. Our findings imply that the use of deep recurrent networks give promising results in that they on average give a more risk-neutral hedging strategy than strategies informed by the Black-Scholes model, although issues in variability were also found.

2 Acknowledgements

I would like to thank Dr Jonathan Shock for agreeing to supervise me on this project, for asking thought-provoking questions, and for offering guidance on the process of scientific writing.

I would also like to thank Jake Stangroom, who helped with path signature and VAE related questions, even when there was no obligation on his part to do so.

Special thanks must be given to Mr Yuri Robbertze, who went beyond the role of supervisor in helping me understand the topics involved in the project, and who's infectious zeal for all things maths continues to inspire me.

I would also like to express gratitude to the scientific community as a whole, whose publicly available publications and articles make learning so accessible, and whose resultant tools (such as ChatGPT) aid considerably in the learning and writing of scientific content.

Finally, and as always, I thank my mom and dad for their continued emotional and financial support. Without them, I would never have been able to pursue my love for mathematics at a tertiary level, and for that I am eternally grateful.

3 Introduction

In 1973, Fischer Black, Myron Scholes and Robert Merton devised a new and revolutionary model to price financial options (Black and Scholes 1973). They proved that under certain conditions, one can perfectly hedge an option by buying and selling the underlying asset. The model posits that the price of stocks follows a geometric brownian motion with constant drift and volatility. As with many models, key assumptions were made that fail in reality. Amongst others, these assumptions included that the market is frictionless (no transaction costs), and that assets can be traded continuously in time.

In reality assets cannot be traded continuously, and thus our trading strategy needs to somehow compensate for this fact, something which strategies informed by the Black-Scholes equations do not do.

This paper aims to assert whether DNNs can better manage portfolios containing the assets underlying a European Best-of option so as to hedge said option. More explicitly, can a neural network govern a trading strategy which will more closely track the value of a contingent claim than a trading strategy as motivated by the Block-Scholes equations. Due to the manner in which neural networks can infer complex relationships without explicit models of the underlying process, we hypothesise that by removing explicit model assumptions and the assumption of continuous trading, a neural network may be able to better hedge a financial instrument in the discrete-time case given that it does not have the restrictive assumptions of the Black-Scholes model.

We also hypothesise that Recurrent Neural Networks will be better able to tackle this task than more standard Feed-Forward Networks, as they have memory mechanisms which is useful for any sequential task, and thus relevant to our task as we are dealing with options based on the time-series data of stocks.

We would like to employ deep learning techniques in order to hedge a financial instrument. A big hurdle to overcome is the fact that there is only one historical realisation of a particular financial instrument, and deep-learning techniques require large amounts of data to perform well. To overcome this we will use a generative model to learn the underlying distribution of the instrument of interest and then generate many novel sample paths by sampling from this learnt distribution, thus enabling us to use more conventional deep learning techniques for the problem of hedging.

An added benefit of the generation process is that we can determine how well our model does on pseudo-real unseen data which can give us an indication on the robustness of our deep hedger under possible future market conditions.

The specific option we will be investigating is an Exotic European Best-of Basket Option of the form:

$$H = 100H^* \tag{1}$$

$$H^* = \max\left\{\frac{S_T^1}{S_0^1}, \frac{S_T^2}{S_0^2}\right\}. \tag{2}$$

where S_t^1 and S_t^2 represent the prices of two stocks at a given time t.

4 Literature Review

4.1 Path Signatures

A path signature is an infinite dimensional tensor containing iterated path integrals over the domain of multidimensional paths (sequential data) (Chevyrev and Kormilitzin 2016). It helps capture complicated interactions between paths, as well as fundamental characteristics of individual paths, such as when events took place relative to one another, whilst not being influenced by the parameterisation of the paths (Lyons and McLeod 2023). It can be viewed as a feature map of a stream of data, and is essentially unique for a given path (Lyons and McLeod 2023, H. Boedihardjo, Ni, and Qian 2013, Horatio Boedihardjo et al. 2014). Being a uniquely determined feature map, a natural question is to ask whether signatures can be used in a machine learning context, where feature extraction allows for more effective and efficient inference. Indeed they have been used in multiple machine learning problems with optimistic results. One successful application has been their use in classifying Chinese Characters, where the incorporation of terms from the path signature as features to a deep convolutional neural network (CNN) showed significant improvement in predictive capabilities (Graham 2013).

4.1.1 Paths

Definition 4.1. A d-dimensional path $X : [a, b] \to \mathbb{R}^d$ is a continuous mapping from an interval of $\mathbb{R}, [a, b]$, to $\mathbb{R}^d$. Where X_t denotes $X(t)$ for $t \in [a, b]$.

Alternatively we can view a d-dimensional path as a collection:

$$X_t = (X_t^1, X_t^2, ..., X_t^d),$$

where $X^i : [a, b] \to \mathbb{R}$ is a one-dimensional path for $\forall i \in \{1, ..., d\}$.

Definition 4.2. Given a one-dimension path $X : [a, b] \to \mathbb{R}$, the *length* of X is denoted

$$|X| := \sup_{P \subset [a,b]} \sum_{i=1}^{n} |X_{t_i} - X_{t_{i-1}}| \tag{3}$$

where the supremum is taken over all finite partitions $P = \{x_1, ..., x_n\}$ in $[a, b]$.

A d-dimensional path $X = (X^1, X^2, ..., X^d)$, is said to be of *bounded/finite variation* if $|X^i|$ is finite $\forall i \in \{1, ..., d\}$.

Definition 4.3. Given a path of bounded variation $X : [a, b] \to \mathbb{R}$ and a function $f : \mathbb{R} \to \mathbb{R}$, the integral of f with respect to X over $[a, b]$ is defined by:

$$\int_a^b f(X_t) dX_t = \int_a^b f(X_t) \frac{dX_t}{dt} dt. \tag{4}$$

4.1.2 Path Signatures

Definition 4.4. Given a d-dimensional path of bounded variation, $X : [a, b] \to \mathbb{R}^d$, and recalling that for $\forall i \in \{1, ..., d\}$, $X^i : [a, b] \to \mathbb{R}$ is a one-dimensional path. For all $i \in \{1, ..., d\}$ we define the integral

$$S(X)^i_{a,t} = \int_{a<s<t} dX_s^i = X_t^i - X_a^i. \tag{5}$$

Similarly, for any pair $i, j \in \{1, ..., d\}$ we define the double integral

$$S(X)^{i,j}_{a,t} = \int_{a<s<t} S(X)^{i}_{a,s} dX^j_s = \int_{a<s<t} \int_{a<r<s} dX^i_r dX^j_s = \int_{a<r<s<t} dX^i_r dX^j_s. \tag{6}$$

Continuing recursively, for any $k \geq 1$, and $i_1, i_2, ..., i_k \in \{1, ..., d\}$, we define the iterated integral

$$S(X)^{i_1,...,i_k}_{a,t} = \int_{a<s<t} S(X)^{i_1,...,i_{k-1}}_{a,t} dX^{i_k}_s. \tag{7}$$

The signature of a path $X : [a, b] \to \mathbb{R}^d$, denoted $S(X)_{a,b}$, is defined to be the infinite collection of all the iterated integrals of X. Formally, $S(X)_{a,b}$ is the sequence of real numbers

$$S(X)_{a,b} = \left(1, S(X)^1_{a,b}, ..., S(X)^d_{a,b}, S(X)^{1,1}_{a,b}, S(X)^{1,2}_{a,b}, ...\right) \tag{8}$$

where the superscripts iterate over all the multi-indexes

$$W = \{(i_1, ..., i_k) | k \geq 1, i_1, ..., i_k \in \{1, ..., d\}\}. \tag{9}$$

4.1.3 Truncated Signatures

In practice we use a truncated form of the signature to some specified depth, where a signature taken to depth N includes all terms up to and including those of N iterated integrals.

$$S(X)^{\leq N}_{a,b} = \left(1, ..., S(X)^{N,...,N}_{a,b}\right) \tag{10}$$

4.1.4 Formal Power Series

Let $e_1, ..., e_d$ be formal indeterminates. The algebra of formal power series in d indeterminates is the vector space of all series of the form

$$\sum_{k=0}^{\infty} \sum_{i_1,...,i_k \in \{1,...,d\}} \lambda_{i_1,...,i_k} e_{i_1} ... e_{i_k}$$

where the second summation runs over all multi-indexes in W as above, and $\lambda_{i_1,...,i_k} \in \mathbb{R}$. A (non-commuting) formal polynomial is a formal power series for which only a finite number of coefficients$\lambda_{i_1,...,i_k}$ are non-zero. The terms $e_{i_1} ... e_{i_k}$ are called monomials. The space of formal power series is frequently termed the tensor algebra of $\mathbb{R}^d$.
The tensor product $\otimes$ of monomials is defined as

$$e_{i_1} ... e_{i_k} \otimes e_{j_1} ... e_{j_l} = e_{i_1} ... e_{i_k} e_{j_1} ... e_{i_l}.$$

We can thus express the signature of a path X by a formal power series where the coefficient of each monomial $e_{i_1} ... e_{i_k}$ is defined to be $S(X)^{i_1,...,i_k}_{a,b}$, and use the same symbol $S(X)_{a,b}$ to denote this representation

$$S(X)_{a,b} = \sum_{k=0}^{\infty} \sum_{i_1,...,i_k \in \{1,...,d\}} S(X)^{i_1,...,i_k}_{a,b} e_{i_1} ... e_{i_k}.$$

4.1.5 Log-signatures

For a power series

$$x = \sum_{k=0}^{\infty} \sum_{i_1,\ldots,i_k \in \{1,\ldots,d\}} \lambda_{i_1,\ldots,i_k} e_{i_1} \ldots e_{i_k}$$

where $\lambda_0 > 0$, the logarithm of x is defined as

$$\log x = \log(\lambda_0) + \sum_{n \geq 1} \frac{-1^n}{n} \left(1 - \frac{x}{\lambda_0}\right)^{\otimes n}$$

where $\otimes n$ denotes the n-th power with respect to the product $\otimes$.

For a path $X : [a, b] \to \mathbb{R}^d$ we define the log signature of X to be the logarithm of the formal power series $S(X)_{a,b}$, $\log S(X)_{a,b}$. The log-signature offers an invertible transformation Zhou 2019 which drastically reduces the dimensionality of the feature space, and hence we will be using the log-signature rather than the signature as inputs to our market generator.

4.2 Stochastic Processes

The structure and definitions that follow in the Stochastic Processes, Stochastic Calculus, Trading and Financial Options sections were informed by the works of Mavuso 2018, Mavuso 2019a, Mavuso 2019b and Mavuso 2019c

Definition 4.5. We term $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ a filtered probability space when $(\Omega, \mathcal{F}, \mathbb{P})$ is a probability space and $\mathbb{F} = \{\mathcal{F}_t : t \in \mathbb{I}\}$ is a sequence of increasing sub σ-algebras of $\mathcal{F}$. Generally $\mathbb{I}$ represents a time-index set and $\mathcal{F}_t$ can be interpreted as being the information available to us at time t.

Definition 4.6. For a given probability space $(\Omega, \mathcal{F}, \mathbb{P})$, we term $Y : \Omega \to \mathbb{R}$ a random variable if Y is $\mathcal{F}$-measurable (i.e.: $Y^{-1}(B) \in \mathcal{F}$ for $\forall B \in \mathcal{B}$ where $\mathcal{B}$ is the Borel σ-algebra.) A stochastic process $X = \{X_t : t \in \mathbb{I}\}$ is collection of random variables, and we say that the X is adapted to the filtration $\mathbb{F}$ if X_t is $\mathcal{F}_t$-measurable for $\forall t \in \mathbb{I}$.

Definition 4.7. Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space and $X = \{X_t : t \in \mathbb{I}\}$ a stochastic process. We term X an $(\mathbb{F}, \mathbb{P})$ martingale if

1. X is adapted to $\mathbb{F}$,
2. $\mathbb{E}(|X_t|) < \infty$ for $\forall t \in \mathbb{I}$,
3. $\mathbb{E}(X_t|\mathcal{F}_s) = X_s \quad \mathbb{P} - a.s.$ for $s \leq t$

A stochastic process satisfying 1 and 2 and in addition satisfying

$$\mathbb{E}(X_t|\mathcal{F}_s) \geq X_s \quad (\text{resp.} \quad \mathbb{E}(X_t|\mathcal{F}_s) \leq X_s) \quad \mathbb{P} - a.s. \quad \text{for} \quad s \leq t$$

is known as a sub (resp. super) martingale.

Definition 4.8. Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space, $X = \{X_t : t \in \mathbb{N}\}$ a stochastic process and $\varphi = \{\varphi_t : t \in \mathbb{N}\}$ a predictable process (i.e.: φ_t is F_{t-1}-measurable for $\forall t$. Then the *martingale transform* of X by φ is the stochastic process $(\varphi \bullet X) = \{(\varphi \bullet X)_t : t \in \mathbb{N}\}$ where

$$(\varphi \bullet X)_0 := 0$$

$$(\varphi \bullet X)_t = \sum_{k=1}^{t} \varphi_k (X_k - X_{k-1}), t \geq 1.$$

Theorem 1. Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space, $X = \{X_t : t \in \mathbb{N}\}$ a stochastic process and $\varphi = \{\varphi_t : t \in \mathbb{N}\}$ a bounded predictable process, then if X is a martingale, $(\varphi \bullet X)$ is also a martingale.

4.3 Stochastic Calculus

This section lays the preliminary foundations required to be able to understand the formulation of the Black-Scholes model against which we will be comparing our hedging strategies.

4.3.1 Brownian Motion

Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space. A stochastic process $W = \{W_t : t \geq 0\}$, is called a *Brownian motion* (BM) w.r.t $\mathbb{F}$ if it satisfies the following properties:

1. $W_0 = 0 \quad \mathbb{P}$ a.s.
2. W_t has independent increments; i.e., for every $0 \leq t_1 < t_2 < \cdots < t_n$, the random variables
$$W_{t_1} - W_{t_0}, W_{t_2} - W_{t_1}, W_{t_n} - W_{t_{n-1}} \tag{11}$$
are independent.
3. W_t has stationary, normally distributed increments, namely:
$$W_t - Ws \sim \mathcal{N}(0, t - s) \tag{12}$$
for $s < t$.
4. $W_t - W_s$ is independent of $\mathcal{F}_s$ for $s < t$, that is:
$$(W_t - W_s) \, | \mathcal{F}_s = W_t - W_s. \tag{13}$$

Similarly, we can define a d-dimensional Brownian Motion $W = (W^1, ..., W^d)$ to be a multivariate stochastic process, whose marginals $W^i : i \in \{1, ..., d\}$ are each Brownian Motions, and whose increments are multivariate Gaussian of the form:

$$W_t - W_s = \begin{pmatrix} W_t^1 - W_s^1 \\ . \\ . \\ . \\ W_t^d - W_s^d \end{pmatrix} \sim \text{MV}\mathcal{N}(\mathbf{0}, \mathbf{\Sigma}) \tag{14}$$

where $\mathbf{\Sigma}_{ii} = t - s$ and $\mathbf{\Sigma}_{ij} = Cov(W_t^i - W_s^i, W_t^j - W_s^j)$.

Proposition 1. Let W^i and W^j be a 2-dimensional Brownian Motion on some filtered probability space and $s < t$, then

$$Cov(W_t^i - W_s^i, W_t^j - W_s^j) = \rho \sqrt{Var(W_t^i - W_s^i)} \sqrt{Var(W_t^j - W_s^j)} \tag{15}$$
$$= \rho(t - s) \tag{16}$$

where ρ is the correlation co-efficient between the increments of W^i and the increments of W^j.

4.3.2 Itô Processes

An adapted process $X = \{X_t : 0 < t < T\}$ is an *Itô Process* if there exists stochastic processes $\mu = \{\mu_t : 0 < t < T\}$ and $\sigma = \{\sigma_t : 0 < t < T\}$ such that

$$X_t = X_0 + \int_0^t \mu_s ds + \int_0^t \sigma_s dW_s \tag{17}$$

often written in differential notation as

$$dX_t = \mu_t ds + \sigma_t dW_t \tag{18}$$

where μ is known as the *drift* and σ the *diffusion coefficient* of X.

Proposition 2. An Itô Process X is a martingale if and only if its drift $\mu = 0$

Proposition 3. Let W be a brownian motion. We state the following identities without proof:

1. $(dt)^2 = 0$
2. $dtdW = 0$
3. $(dW)^2 = dt$

4.3.3 Itô's Lemma

Let $f : [0, \infty) \times \mathbb{R} \to \mathbb{R}$ be a function s.t $\frac{\partial f}{\partial t}$, $\frac{\partial f}{\partial x^i}$ and $\frac{\partial^2 f}{\partial x^i x^j}$ all exist and are continuous for every $i, j \in \{1, ..., d\}$. Furthermore, suppose that each X^i for $i \in \{1, ..., d\}$ is an Itô process satisfying

$$dX_t^i = \mu_t^i dt + \sigma_t^i dW_t^i, \tag{4.3.19}$$

then

$$dY_t = df(t, X_t) = \frac{\partial f}{\partial t} dt + \sum_{i=1}^{d} \frac{\partial f}{\partial X_t^i} dX_t + \frac{1}{2} \sum_{i=1}^{d} \sum_{j=1}^{d} \frac{\partial^2 f}{\partial X_t^i X_t^j} (dX_t^i)(dX_t^j) \tag{4.3.20}$$

4.3.4 Example: Geometric Brownian Motion

Consider the following SDE:

$$dX_t = \mu X_t dt + \sigma X_t dW_t, \qquad \mu \in \mathbb{R}, \sigma \in (0, \infty)$$

We would like to solve this explicitly by finding X_t in terms of BM W_t. Applying Itô's Lemma to $Y_t = f(t, X_t) = ln(X_t)$ and noting that

$$\frac{\partial f}{\partial t} = 0, \quad \frac{\partial f}{\partial X_t} = \frac{1}{X_t}, \quad \frac{\partial^2 f}{\partial X_t^2} = -\frac{1}{X_t^2} \tag{4.3.21}$$

gives

$$dY_t = \frac{1}{X_t} dX_t - \frac{1}{2X_t^2} (dX_t)^2 \tag{4.3.22}$$

$$= \frac{1}{X_t} (\mu X_t dt + \sigma X_t dW_t) - \frac{1}{2X_t^2} (\mu X_t dt + \sigma X_t dW_t)^2 \tag{4.3.23}$$

Expanding, using 4.3.2 for simplification, and grouping yields:

$$dY_t = \left(\mu - \frac{1}{2}\sigma^2\right) dt + \sigma dW_t. \tag{4.3.24}$$

Integrating both sides, we obtain:

$$Y_t = Y_0 + \left(\mu - \frac{1}{2}\sigma^2\right) t + \sigma W_t \tag{4.3.25}$$

$$\Rightarrow Y_t \sim \mathcal{N}\left(\left(Y_0 + \mu - \frac{1}{2}\sigma^2\right), \sigma^2 t\right) \tag{4.3.26}$$

Finally, recalling that $Y_t = \log(X_t)$, exponentiating 4.3.25 gives:

$$X_t = X_0 \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right) t + \sigma W_t\right) \tag{4.3.27}$$

which shows that X_t follows a log-normal distribution.

4.4 Trading

We will be focusing on trading in discrete time. In this context, let $T \in \mathbb{N}$ be the point in time up to which we consider trades/values of our portfolio. Assume we are working with $d+1$ assets. The prices of these $d+1$ assets are given by the stochastic processes $(B, S) = (B, S^1, ..., S^d)$, where B represents the value of our bank account (also known as the *riskless* asset), and S^i represents the value of some financial asset.

Definition 4.9. Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space and assume that (B, S) is adapted to the filtration $\mathbb{F}$, then we term the tuple $((\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P}), (B, S))$ a *market*.

Definition 4.10. Let N be an asset in our market. We can define all prices of our market (B, S) as the discounted prices $(Y, X) = (\frac{B}{N}, \frac{S}{N})$ where $X = (X^1, ..., X^d)$ is defined such that $X^i = \frac{S^i}{N}$. When such an asset is used for discounting it is termed a *numeraire*. We will see how the use of numeraires can simplify calculations significantly.

Often B is chosen as the numeraire, resulting in our discounted prices being $(1, X)$. The measure under this numeraire is known as the *risk-neutral measure*, denoted $\mathbb{P}^*$, whilst the measure under no numeraire is known as the *real-world measure*, denoted $\mathbb{P}$.

Definition 4.11. Let φ_i^i be the number of units of asset i held between time $t-1$ to time t and ν^i the number of units of the riskless asset held over the same period. Clearly we need to be able to know each of these holdings at time $t-1$, thus we define a *trading strategy* to be the predictable process $(\nu, \varphi) = (\nu, \varphi^1, ..., \varphi^d)$.

The *value* of our portfolio under the trading strategy(ν, φ) is thus the stochastic process $V((\nu, \varphi)) = \{V_t(\nu, \varphi) : t = 0, 1, ..., T\}$ defined by

$$V_0((\nu, \varphi)) = (\nu_1, \varphi_1) \cdot (Y_0, X_0) = \nu_1 Y_0 + \sum_{i=1}^{d} \varphi_1^i X_0^i$$

and

$$V_t((\nu,\varphi)) = (\nu_t,\varphi_t)\cdot(Y_t,X_t) = \nu_t Y_t + \sum_{i=1}^{d}\varphi_t^i X_t^i \quad t \geq 1.$$

The *gains* associated with a strategy (ν,φ) is the stochastic process $G(\varphi) = \{G_t(\varphi) : t = 0,1,...,T\}$ where

$$G_0(\varphi) := 0$$
$$G_t(\varphi) = \sum_{k=1}^{t}\varphi_k\cdot(X_k - X_{k-1}) \quad t \geq 1$$

Note that gains are only associated with changes in the non-riskless assets $X^1,...X^d$, thus we see that by knowing the initial value of the portfolio and $\varphi^1,...,\varphi^d$ we can in fact recover the amount that must be invested in the riskless account via the formula

$$V_0((\nu,\varphi)) + G_t(\varphi) = \nu_t + \sum_{i=1}^{d}\varphi_t^i X_t^i.$$

Thus, specifying the initial value v_0 and the predictable holdings of the non-riskless assets $\varphi = (\varphi^1,...\varphi^d)$ uniquely specifies the full strategy. Consequently, we subsequently denote strategies by (v_0,φ).

Definition 4.12. A trading strategy φ is termed an arbitrage strategy if

$$V_0(\varphi) = 0, V_T(\varphi) \geq 0 \quad \mathbb{P}\text{-a.s. and } \mathbb{P}(V_T(\varphi) > 0) > 0.$$

A market model is called arbitrage free if there does not exist any arbitrage strategies.

Definition 4.13. Let $(\Omega,\mathcal{F},\mathbb{P})$ be a measure space and X a random vector. A probability measure $\mathbb{P}^*$ on $(\Omega,\mathcal{F})$ is termed an *Equivalent Martingale Measure* (EMM) for X if

1. $\mathbb{P}^*$ is equivalent to $\mathbb{P}$, i.e.:

$$\mathbb{P}^*(A) = 0 \Leftrightarrow \mathbb{P}(A) = 0 \quad \text{for} \quad \forall A \in \mathcal{F}$$

2. X is an $(\mathbb{F},\mathbb{P}^*)$ -martingale.

We denote $\mathcal{P}$ to be the set of EMMs for X, and $\mathbb{E}^*$ to be the expectation w.r.t $\mathbb{P}^*$

Theorem 2. (FTAP1). For a financial market $(\Omega,\mathcal{F},\mathbb{F},\mathbb{P}),X)$, the following are equivalent:

1. There are no arbitrage opportunities;
2. $\mathcal{P} \neq \emptyset$.

4.5 Financial Options

Definition 4.14. A *contingent claim* H is an $\mathcal{F}_T$-measurable random variable. We term a contingent claim H a *derivative* of some underlying asset X if H is measurable with respect to $\mathcal{F}_T^X$.

Issuers of contingent claims view them as liabilities that expire at time T. Thus in order to safeguard against the stochastic outcomes of H, the issuer would like to *replicate* the price of the contingent claim by trading the assets underlying the claim.

Definition 4.15. A strategy φ s.t $V_T(\varphi) = H \quad \mathbb{P}$-a.s. is termed a *replicating strategy/portfolio.* If such a strategy exists for a given contingent claim H, then we term H *attainable.* A market is said to be *complete* if every contingent claim is attainable.

Theorem 3. (FTAP 2). Given a financial market $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P}), X)$ with no arbitrage opportunities, the following are equivalent:

1. The market is complete
2. $|\mathcal{P}| = 1$, i.e. there is a unique EMM.
3. X has the predictable representation property (PRP) with respect to every $P^* \in \mathcal{P}$. That is, every $(\mathbb{F}, \mathbb{P}^*)$-martingale M with $M_0 = 0$ can be written as a martingale transform w.r.t X.

Definition 4.16. European Call Option.
Let S be a stock and S_t the value of the some stock at time t. A *European call option* is a derivative of S of the form

$$H = (S_T - K)^+$$

where K is known as the strike price of the option. What this option represents is contract whereby the purchaser of the option has the right to buy the underlying stock for price K at time T, generally done when $S_T \geq K$, and consequently meaning that the effective value of the option is given by H. European options differ from American options in that they can only be exercised at maturity (time T).

Definition 4.17. A *best of European option* is an option of the form

$$H = \max\{H_T^1, H_T^2\}$$

where H^1 and H^2 are functions of two stocks S^1 and S^2.

For example:

$$H = \max\left\{\frac{S_T^1}{S_0^1}, \frac{S_T^2}{S_0^2}\right\}$$

is the option whose payoff corresponds to the maximum of the increases in prices of stocks 1 and 2 relative to their initial prices.

4.6 The Black-Scholes Model

For a given market (B, X) the Black-Scholes model states that the prices of the financial assets $X = (X^1, ..., X^d)$ follow that of a multivariate Geometric Brownian Motion process:

$$dX_t^i = \mu_i X_t^i dt + \sigma_i X_t^i dW_t^i, \qquad \mu_i \in \mathbb{R}, \quad \sigma_i \in (0, \infty), \quad i \in \{1, ..., d\} \tag{4.6.1}$$

where the Brownian Motions have increments which are Multivariate Normal:

$$W_t - W_s = \begin{pmatrix} W_t^1 - W_s^1 \\ . \\ . \\ . \\ W_t^d - W_s^d \end{pmatrix} \sim \text{MV}\mathcal{N}(\mathbf{0}, \mathbf{\Sigma}) \tag{4.6.2}$$

where $\mathbf{\Sigma}_{ii} = t - s$ and $\mathbf{\Sigma}_{ij} = \rho_{ij}(t - s)$.

4.7 Option Pricing

Definition 4.18. Let $((\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P}), (\frac{B}{N}, \frac{S}{N})$ be a complete market being discounted by some numeraire $N \in (B, S)$. We assume that there are no arbitrage opportunities in this market. Let H_t represent the price of some contingent claim H at time t. We define the discounted price of the option at time t to be:

$$\frac{H_t}{N_t} := \mathbb{E}^N \left[\frac{H_T}{N_T} | \mathcal{F}_t \right] \tag{4.7.1}$$

where $\mathbb{E}^N$ is the expectation as taken under the EMM making $(\frac{B}{B}, \frac{S}{N}))$ a martingale, whose existence is guaranteed by the FTAP 2 4.5.

Equivalently, since N_t is $\mathcal{F}_t$-measurable, we can write this as:

$$H_t = \mathbb{E}^N \left[\frac{N_t}{N_T} H_T | \mathcal{F}_t \right] \tag{4.7.2}$$

Proposition 4. Under the appropriate EMM, if $\frac{X_t}{N_t}$ is an Itô Process then by 4.5 and 4.3.2 it has drift = zero.

4.7.1 Pricing under Black-Scholes

We now use the frameworks of stochastic calculus and the Black-Scholes model to show how one finds the price (and consequently the Black-Scholes informed hedging strategy) of a specific European Basket Option, from which we will be able to directly get the price of our actual option as described in 3.

Let $((\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P}), (\frac{B}{N}, \frac{S}{N})$ be a complete market being discounted by some numeraire $N \in (B, S)$. Consider a contingent claim of the form

$$H_T = \max\left\{X_T^1, X_T^2\right\} = X_T^1 \mathbb{I}_{\left\{X_T^1 \geq X_T^2\right\}} + X_T^2 \mathbb{I}_{\left\{X_T^2 \geq X_T^1\right\}}. \tag{4.7.3}$$

For notational ease set: $X_t^{\frac{1}{2}} = \frac{X_t^1}{X_t^2}$, then 4.7.3 becomes:

$$H_T = X_T^1 \mathbb{I}_{\left\{1 \geq X_T^{\frac{2}{1}}\right\}} + X_T^2 \mathbb{I}_{\left\{1 \geq X_T^{\frac{1}{2}}\right\}} \tag{4.7.4}$$

Using definition 4.7 we have that

$$H_t = \mathbb{E}^N \left[\frac{N_t}{N_T} H_T | \mathcal{F}_t \right] \tag{4.7.5}$$

$$= \mathbb{E}^N \left[\frac{N_t}{N_T} X_T^1 \mathbb{I}_{\left\{1 \geq X_T^{\frac{2}{1}}\right\}} + \frac{N_t}{N_T} X_T^2 \mathbb{I}_{\left\{1 \geq X_T^{\frac{1}{2}}\right\}} | \mathcal{F}_t \right] \tag{4.7.6}$$

$$= \mathbb{E}^N \left[\frac{N_t}{N_T} X_T^1 \mathbb{I}_{\left\{1 \geq X_T^{\frac{2}{1}}\right\}} | \mathcal{F}_t \right] + \mathbb{E}^N \left[\frac{N_t}{N_T} X_T^2 \mathbb{I}_{\left\{1 \geq X_T^{\frac{1}{2}}\right\}} | \mathcal{F}_t \right] \tag{4.7.7}$$

$$= \mathbb{E}^1 \left[\frac{X_t^1}{X_T^1} X_T^1 \mathbb{I}_{\left\{1 \geq X_T^{\frac{2}{1}}\right\}} | \mathcal{F}_t \right] + \mathbb{E}^2 \left[\frac{X_t^2}{X_T^2} X_T^2 \mathbb{I}_{\left\{1 \geq X_T^{\frac{1}{2}}\right\}} | \mathcal{F}_t \right] \tag{4.7.8}$$

$$= X_t^1 \mathbb{E}^1 \left[\mathbb{I}_{\left\{1 \geq X_T^{\frac{2}{1}}\right\}} | \mathcal{F}_t \right] + X_t^2 \mathbb{E}^2 \left[\mathbb{I}_{\left\{1 \geq X_T^{\frac{1}{2}}\right\}} | \mathcal{F}_t \right] \tag{4.7.9}$$

$$= X_t^1 \mathbb{Q}^1 \left(\left\{ 1 \geq X_T^{\frac{2}{1}} \right\} | \mathcal{F}_t \right) + X_t^2 \mathbb{Q}^2 \left(\left\{ 1 \geq X_T^{\frac{1}{2}} \right\} | \mathcal{F}_t \right) \tag{4.7.10}$$

Where between line 4.7.7 and 4.7.8 we have performed a change of numeraire, and so $\mathbb{E}^i$ represents the expectation taken under the EMM $\mathbb{Q}^i$ making all the assets discounted by X_t^i martingales.

Thus if we can evaluate the probabilities $\mathbb{Q}^1$ and $\mathbb{Q}^2$ then not only can we calculate the price of the contingent claim at time t, but these probabilities would also give exactly the portfolio holdings of X_t^1 and X_t^2 needed to perfectly replicate the value of the claim H (if we could trade continuously).

Now suppose X_t^i and X_t^j represent the price of two stocks as described under the Black-Scholes model:

$$dX_t^i = \mu^i X_t^i dt + \sigma^i X_t^i dW_t^i, \quad \mu \in \mathbb{R}, \sigma \in (0, \infty) \quad i \in \{1, 2\}. \tag{4.7.11}$$

We wish to find the distribution of $X_t^{\frac{i}{j}}$ in order to evaluate the probability in 4.7.10.

Setting $Y_t = f\left(t, X_t^1, X_t^2\right) = X_t^{\frac{i}{j}}$, applying 4.3.20, and subsequently subbing in 4.7.11 we find

$$dY_t = 0 + \frac{1}{X_t^j} dX_t^i - \frac{X_t^i}{(X_t^j)^2} dX_t^j + \frac{1}{2} \left(0 (dX_t^i)^2 + \frac{2X_t^i}{(X_t^j)^3} (dX_t^j)^2 - \frac{2}{(X_t^j)^2} (dX_t^i)(dX_t^j) \right) \tag{4.7.12}$$

$$= \frac{1}{X_t^j} \left(\mu^i X_t^i dt + \sigma^i X_t^i dW_t^i \right) - \frac{X_t^i}{(X_t^j)^2} \left(\mu^j X_t^j dt + \sigma^j X_t^j dW_t^j \right)$$

$$+ \frac{X_t^i}{(X_t^j)^3} \left(\sigma_j^2 (X_t^j)^2 dt \right) - \frac{1}{(X_t^i)^2} \left(\sigma_i \sigma_j X_t^i X_t^j \rho dt \right) \tag{4.7.13}$$

$$= X_t^{\frac{i}{j}} \left[(\mu^i - \mu^j + \sigma_j^2 - \sigma_i \sigma_j \rho) dt + \sigma_i dW_t^i - \sigma_j dW_t^j \right] \tag{4.7.14}$$

If we now consider this process under the appropriate EMM $\mathbb{Q}^j$, then the drift of our discounted process $\frac{X_t^i}{X_t^j}$ goes to zero so we are simply left with:

$$dX_t^{\frac{i}{j}} = X_t^{\frac{i}{j}} \left[\sigma_i dW_t^i - \sigma_j dW_t^j\right] \tag{4.7.15}$$

This looks promising, as it resembles the Black-Scholes formulation for a single stock, however we need to do some further manipulation to get it it relative to a single BM process.

For notational simplification, set $X_t = X_t^{\frac{i}{j}}$.
Setting $Y_t = ln\,(X_t)$, applying Itos Lemma, substituting 4.7.15 and finally simplifying using 4.3.2 yields:

$$dY_t = \frac{1}{X_t} dX_t - \frac{1}{2(X_t)^2}(dX_t)^2 \tag{4.7.16}$$

$$= \sigma_i dW_t^i - \sigma_j dW_t^j - \frac{1}{2}\left(\sigma_i^2 + \sigma_j^2 - 2\sigma_i\sigma_j\rho\right) dt \tag{4.7.17}$$

We set $(\sigma^*)^2 = \sigma_i^2 + \sigma_j^2 - 2\sigma_i\sigma_j\rho$.

Claim 1. We claim that $W_t = \frac{\sigma_i W_t^i - \sigma_j W_t^j}{\sigma^*}$ is a Brownian Motion.

Proof. 1. Clearly, $W_0 = 0$ a.s. since $W_0^i = W_0^j = 0$ a.s.

2. Similarly, W_t has stationary and independent increments.

3. Under the Black-Scholes model, the increments of Brownian motion are multivariate normal, which implies that linear combinations of the increments are themselves Gaussian. Thus,

$$W_t - W_s = \frac{1}{\sigma^*}\left(\sigma_i(W_t^i - W_s^i) - \sigma_j(W_t^j - W_s^j)\right) \sim \mathcal{N}(\mu, \sigma^2),$$

and we have that

$$\begin{aligned}\mu &= \mathbb{E}(W_t - W_s) \\ &= \frac{1}{\sigma^*}\left(\sigma_i\mathbb{E}(W_t^i - W_s^i) - \sigma_j\mathbb{E}(W_t^j - W_s^j)\right) \\ &= \frac{0+0}{\sigma^*} = 0,\end{aligned} \tag{4.7.18}$$

and

$$\begin{aligned}\sigma^2 &= \mathrm{Var}(W_t - W_s) \\ &= \frac{1}{(\sigma^*)^2}\left[\sigma_i^2\mathrm{Var}(W_t^i - W_s^i) + \sigma_j^2\mathrm{Var}(W_t^j - W_s^j) + 2\mathrm{Cov}\left(\sigma_i(W_t^i - W_s^i), -\sigma_j(W_t^j - W_s^j)\right)\right] \\ &= \frac{1}{(\sigma^*)^2}\left[\sigma_i^2(t-s) + \sigma_j^2(t-s) - 2\sigma_i\sigma_j\mathrm{Cov}(W_t^i - W_s^i, W_t^j - W_s^j)\right] \\ &= \frac{1}{(\sigma^*)^2}\left[(t-s)(\sigma_i^2 + \sigma_j^2 - 2\sigma_i\sigma_j\rho)\right] \\ &= (t-s).\end{aligned} \tag{4.7.19}$$

Therefore, $W_t - W_s \sim \mathcal{N}(0, t-s)$. □

Thus we have

$$dY_t = -\frac{1}{2}(\sigma^*)^2 dt + \sigma^* dW_t \tag{4.7.20}$$

$$\Rightarrow \ln(X_T^{\frac{i}{j}}) = Y_T = Y_t + \int_t^T -\frac{1}{2}(\sigma^*)^2 ds + \int_t^T \sigma^* dW_s \tag{4.7.21}$$

$$= Y_t - \frac{1}{2}(\sigma^*)^2(T-t) + \sigma^*(W_T - W_t) \tag{4.7.22}$$

$$\Rightarrow X_T^{\frac{i}{j}} = X_t^{\frac{i}{j}} \exp\left(-\frac{1}{2}(\sigma^*)^2(T-t) + \sigma^*(W_T - W_t)\right) \tag{4.7.23}$$

Returning to 4.7.10 we find

$$\mathbb{Q}^i\left(\left\{1 \geq X_T^{\frac{j}{i}}\right\}|\mathcal{F}_t\right) = \mathbb{Q}^i\left(\left\{0 \geq \ln(X_T^{\frac{j}{i}})\right\}|\mathcal{F}_t\right) \tag{4.7.24}$$

$$= \mathbb{Q}^i\left(\left\{0 \geq \ln(X_t^{\frac{j}{i}}) - \frac{1}{2}(\sigma^*)^2(T-t) + \sigma^*(W_T - W_t)\right\}|\mathcal{F}_t\right) \tag{4.7.25}$$

$$= \mathbb{Q}^i\left(\left\{\frac{W_T - W_t}{\sqrt{T-t}} \leq \frac{\ln X_t^{\frac{i}{j}} + \frac{1}{2}(\sigma^*)^2(T-t)}{\sigma^*\sqrt{(T-t)}}\right\}|\mathcal{F}_t\right) \tag{4.7.26}$$

$$\tag{4.7.27}$$

Recalling that $\frac{W_T - W_t}{\sqrt{T-t}} \sim \mathcal{N}(0,1)$ and that Φ denotes the cumulative distribution function (CDF) for a standard normal, we find that:

$$\mathbb{Q}^i\left(\left\{1 \geq X_T^{\frac{j}{i}}\right\}|\mathcal{F}_t\right) = \Phi\left(\frac{\ln X_t^{\frac{i}{j}} + \frac{1}{2}(\sigma^*)^2(T-t)}{\sigma^*\sqrt{(T-t)}}\right) \tag{4.7.28}$$

$$:= \Phi\left(d_1^{\frac{i}{j}}(t)\right) \tag{4.7.29}$$

For ease of calculation, we define

$$d_2^{\frac{i}{j}}(t) := d_1^{\frac{i}{j}}(t) - \sigma^*\sqrt{T-t} = \frac{\ln X_t^{\frac{i}{j}} - \frac{1}{2}(\sigma^*)^2(T-t)}{\sigma^*\sqrt{(T-t)}}$$

and note that $d_1^{\frac{j}{i}}(t) = -d_2^{\frac{i}{j}}(t)$.

Thus, our final formula for H_t is

$$H_t = X_t^1 \Phi\left(d_1^{\frac{i}{j}}(t)\right) + X_t^2 \Phi\left(-d_2^{\frac{i}{j}}(t)\right) \tag{4.7.30}$$

4.7.2 Estimating σ_i, σ_j and ρ

In order to calculate the holdings in each stock as described above, we need to estimate σ^* for which we need estimates of σ_i, σ_j and ρ. The derivation below motivates our choice of estimator.

Let S_t be the price of our stock at time t and suppose our most recent observation of the stock is at time $t - dt$ (in our specific case $dt = \frac{1}{365}$, i.e. one day).
By Black-Scholes we have that

$$S_t = S_{t-dt}\exp\left[\left(\mu - \frac{1}{2}\sigma^2\right)dt + \sigma\left(W_t - W_{t-dt}\right)\right] \tag{4.7.31}$$

$$\Rightarrow \ln\left(\frac{S_t}{S_{t-dt}}\right) = \left(\mu - \frac{1}{2}\sigma^2\right)dt + \sigma\left(W_t - W_{t-dt}\right) \tag{4.7.32}$$

From this we see that for no matter which t is chosen, $\ln\left(\frac{S_t}{St-dt}\right)$ is identically and independently distributed as long as we consider disjoint intervals.
Taking expectations on both sides and recalling that increments of BM have zero mean:

$$\mathbb{E}\left[\ln\left(\frac{S_t}{S_{t-dt}}\right)\right] = \left(\mu - \frac{1}{2}\sigma^2\right)dt \tag{4.7.33}$$

$$\tag{4.7.34}$$

from which we find that

$$\text{TS2} := \ln\left(\frac{S_t}{S_{t-dt}}\right) - \mathbb{E}\left[\ln\left(\frac{S_t}{S_{t-dt}}\right)\right] = \sigma\left(W_t - W_{t-dt}\right) \tag{4.7.35}$$

and consequently that

$$Var(\text{TS2}) = \sigma^2(dt). \tag{4.7.36}$$

We can easily get an unbiased estimate of 4.7.33 by taking the mean of all log ratio increments over the training set. Consequently, we get an unbiased estimate for 4.7.35 at each t, which allows us to calculate the variance over all t's to get an estimate for $Var(\text{TS2})$. Finally, this allows use to get a point estimate for σ using

$$\sigma = \sqrt{Var(TS2)/dt}$$

Doing this calculation for each time series related to S_t^i and S_t^j gives point estimates for σ_i and σ_j, and recalling 1 we have that

$$\rho = \frac{Cov(TS2^i, TS2^j)}{\sigma_i \sigma_j dt}$$

which gives us a point estimate for ρ and consequently for σ^*.

4.7.3 Our Option

As mentioned, the claim we will be hedging against is of the form

$$H = 100H^*$$

where

$$H^* = \max\left\{\frac{S_T^1}{S_0^1}, \frac{S_T^2}{S_0^2}\right\}.$$

Note that the derivation of H_t^* for all $t \in \{0, ..., T\}$ follows identically that of $H_t = \max\left\{X_t^1, X_t^2\right\}$ as described above, with X_t^i simply being replaced with $\frac{S_t^i}{S_0^i}$ in all positions.

In other words the option gives you the maximum percentage gain of the stocks relative to their initial prices.

We will follow a claim which expires at time $T = 30$, meaning we will have 30 opportunities to re-balance our portfolio at times $t = 0, 1, 2, ..., 29$.

4.8 Neural Networks

Neural networks (also known as Multilayer Perceptrons (MLPs)) are a ubiquitous architecture in the machine learning field. They act as non-linear function approximators and have been employed to solve a plethora of tasks. They have the ability to outperform humans in complex games (such as chess and go), can be used for speech recognition, image classification, facial recognition as well as the powerful and famous generative uses such as diffusion based image generation and transformer based text generation.

4.8.1 Basic feed-forward MLP Architecture

Suppose we have input/explanatory data, $\mathbf{x} \in \mathbb{R}^d$ and output/response data, $\mathbf{y} \in \mathbb{R}^e$. Let $f : \mathbb{R}^d \to \mathbb{R}^e$ be the function relating the two, ie: $f(\mathbf{x}) = \mathbf{y}$. A neural network $f^{\mathcal{NN}} : \mathbb{R}^d \to \mathbb{R}^e$ is a highly parameterised function which approximates f. More formally we have that

$$f^{\mathcal{NN}} = l^{out} \circ l^m \circ ... \circ l^1 \circ l^{in} \tag{4.8.1}$$

where

$$l^{in}(\mathbf{x}) = \sigma\left(\mathbf{W}^{(0,1)}\mathbf{x} + \mathbf{b}^1\right) \tag{4.8.2}$$

$$l^j = \sigma\left(\mathbf{W}^{(j-1,j)}\mathbf{k}^j + \mathbf{b}^j\right) \tag{4.8.3}$$

and

$$\mathbf{k}^j = l^{j-1} \circ ... \circ l^1 \circ l^{in}(\mathbf{x}). \tag{4.8.4}$$

When using a differentiable activation function σ and defining a differentiable loss function which takes as inputs the outputs of the neural network, the neural network can minimise the loss-function via the famous iterative algorithm of the forward pass, backpropogation of gradients, and finally optimisation step (such as Stochastic Gradient Descent (SGD) or Adam). What these techniques do is iteratively update the parameters (weights and biases) of the neural network so as to descend the loss landscape, minimising the objective. These methods are accompanied by appropriate techniques which limit the complexity of the model, known as the process of regularisation. The regularisation of the model is what can be viewed as the "learning", as this is what allows the model to effectively generalise to unseen data.

4.9 Universal Approximation Theorem

A major motivation for the use of neural networks is the fact that they are universal approximators, More specifically, it has been shown that a single layer neural network using a contiunous sigmoidal activation function can arbitrarily approximate any continuous function on some cube. The proof presented here follows that of Cybenko 1989, but with some added details and preliminary results upon which the proof depends.

Let I_n denote the unit cube in $\mathbb{R}^n$, let $C(I_n)$ be the space of continuous functions on I_n taking values in $\mathbb{R}$, and let $\|\cdot\|_\infty$ denote the usual supremum norm on $C(I_n)$. Let $M(I_n)$ denote the space of finite, signed, regular Borel measures on I_n.

Definition 4.19. We term a function $\sigma : \mathbb{R} \to \mathbb{R}$ *discriminatory* if for any $\mu \in M(I_n),\ y \in \mathbb{R}^n,\ \theta \in \mathbb{R}$

$$\int_{I_n} \sigma(y^T x + \theta) d\mu = 0 \tag{4.9.1}$$

$$\implies \mu = 0 \tag{4.9.2}$$

Definition 4.20. We term $\sigma : \mathbb{R} \to \mathbb{R}$ *sigmoidal* if

$$\sigma(t) \to 1 \quad \text{as} \quad t \to \infty \tag{4.9.3}$$

$$\sigma(t) \to 0 \quad \text{as} \quad t \to -\infty \tag{4.9.4}$$

Proposition 5. Any bounded, continuous sigmoidal function σ is discriminatory.

Proof: Cybenko 1989.

4.9.1 The Hahn-Banach Theorem

Let $(V, \|\cdot\|)$ be a normed space over some the field $\mathbb{K} = \mathbb{R}$ or $\mathbb{C}$ and let S be a proper subspace of V, then there exists a linear continuous mapping $L : V \to \mathbb{K}$ such that

$$L(u) = 0 \quad \forall u \in M, \tag{4.9.5}$$

$$L \neq 0 \tag{4.9.6}$$

Proof: Ebobisse 2024.

4.9.2 The Riesz-Markov-Kakutani Representation Theorem

Let X be a locally compact Hausdorff space and let $C_C(X) = \{f : X \to \mathbb{R}, f \text{ continuous}\}$, then for any linear continuous mapping $L : X \to \mathbb{K}$ there exists a unique Radon measure μ on X such that

$$L(f) = \int_X f d\mu \qquad \forall f \in C_C(X) \tag{4.9.7}$$

Proof: ESPEJO 2023.

4.9.3 Universal Approximation Theorem

Consider the normed space $(C(I_n), \|\cdot\|_\infty)$ and let σ be a continuous, sigmoidal function, then the set of functions

$$S = \left\{ G : I_n \to \mathbb{R}, G(x) = \sum_{j=1}^{N} \alpha_j \sigma(y_j^T x + \theta_j),\ y \in \mathbb{R}^n,\ \theta \in \mathbb{R},\ \forall x \in I^n \right\}$$

is dense in $C(I_n)$, i.e.: $\overline{S} = C(I_n)$.

Proof:

We use proof by contradiction to show that $\overline{S} = C(I_n)$.

Clearly $S \subseteq C(I_n)$ and a simple check verifies that it is in fact a subspace of $C(I_n)$.

Assume that $\overline{S} := R$ is a proper subset of $C(I_n)$. Since S is a subspace, this means that R is a proper, closed subspace of $C(I_n)$. By the Hahn-Banach theorem we have that there is a linear continuous functional L on $C(I_n)$ such that $L(R) = L(S) = \{0\}$, with $L \neq 0$.

By the Riesz-Markov-Kakutani Representation Theorem, there is some μ on the Borel σ-algebra of I_n such that

$$L(f) = \int_{I_n} f d\mu \qquad \forall f \in C(I_n).$$

Since I_n is compact and μ a Radon measure, we have that μ is finite on I_n, meaning that $\mu \in M(I_n)$. In particular, for $y \in \mathbb{R}^n,\ \theta \in \mathbb{R}$

$$\int_{I_n} \sigma\left(y^T x + \theta\right) d\mu = 0,$$

since $L(S) = \{0\}$. By assumption σ is a continuous sigmoidal function, and so by 4.9 σ is discriminatory. Thus by the above we have that $\mu = 0$, but then for $\forall f \in C(I_n)$ we have that

$$L(f) = \int_{I_n} f d\mu = 0,$$

contradicting the fact that $L \neq 0$.

This contradiction implies that our initial assumption that R is a proper subspace must be incorrect, i.e.: $\overline{S} = C(I_n)$.

4.10 Variational Auto-Encoders

4.10.1 Auto-Encoders

Auto-Encoders are a class of neural networks which can be used for amongst other things: dimensionality reduction, feature extraction and data compression. They combine an encoder and decoder and attempt to reproduce input data by compressing it down into a lower dimensional latent space via the encoder, and then use this latent representation to reproduce the original input via the decoder. In this way the encoder is forced to extract core features of the data so that the decoder can effectively recover the data from the latent encoding.

This notion of a vectorized latent embedding which captures core information raises interesting questions. Can we take two data samples, get their latent embeddings, linearly interpolate their embeddings and then run this interpolation through the decoder to get a sample which somehow combines the core features of both samples? Can we somehow sample from the latent space to generate wholly novel samples which come from the same distribution as our original data? If we were attempt to do either of these things using a standard Auto-Encoder we would often find that

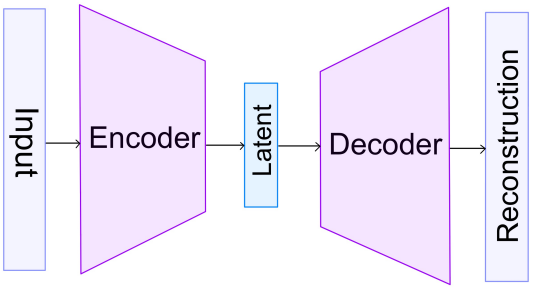


Figure 1: Auto-Encoder Architecture

an issue is the lack of regularity of the latent space. What we mean by this is that there is nothing in the architecture of an Auto-Encoder which, for instance, ensures that the latent space surrounding a particular latent embedding will produce samples "similar" to that latent embedding, or that there is any inherent regular distributional structure to the latent space. Although the Auto-Encoder often cluster similar inputs to similar latent embeddings naturally in order for efficient reproductions, the lack of an explicit enforcing of latent space regularity leads to the issue of i) not having a distribution to sample from and ii) not being able to ensure that the latent space surrounding a specific embedding holds "similar" embeddings, ie: a lack of regularity. These issues are addressed in the modified VAE architecture as described below.

4.10.2 General Description

Variational Auto-Encoders (VAEs) are a class of generative neural networks which consist of an encoder and decoder. The encoder maps the input data into a latent space, typically of lower dimension than that of the input, and the decoder maps latent-embeddings back into the "real-world" space. The encoder can be interpreted as applying a non-linear dimensionality reduction to the input data. The aim of the encoder is to learn a useful latent-representation of the data, where core/informative features are captured in the dimensions of the latent space. A key feature of VAEs over their cousin the Auto-Encoder is that VAEs attempt to force the latent space to be more regular. This can be achieved by penalising the estimated latent space to ensure that it is closer to some prior distribution of our choice. This allows us to sample from the continuous prior after training and consequently pass our samples through the decoder to generate novel data samples that in theory come from the same distribution as our original data.

4.10.3 Formal Description

The formal description that follows is closely based on the work of (Kingma and Welling 2019).

Let $\mathbf{x}$ be a vector representing the k-dimensional data whose joint distribution we would like to model. Let $p^*(\mathbf{x})$ denote the true distribution of X. Our goal is to approximate $p^*(\mathbf{x})$ with a

parameterised model $p_{\boldsymbol{\theta}}(\mathbf{x})$. In other words, we wish to find parameters $\boldsymbol{\theta}$ such that

$$\mathbf{x} \sim p^*(\mathbf{x}) \approx p_{\boldsymbol{\theta}}(\mathbf{x}).$$

It holds that we would like the family of models possible under our parameterisation to be highly flexible, as we do not know how complex the true distribution $p^*(\mathbf{x})$ may be. Neural Networks give such a family of models.

We can then introduce the notion of *latent variables*, denoted $\mathbf{z}$, also known as unobservable variables due to the fact that they are not found explicitly in our dataset. Although they are unobserved, they appear in our model through $\mathbf{x}$'s conditional dependence on them. Their distribution, $p(\mathbf{z})$, is known as the prior distribution over $\mathbf{z}$. Via the law of total probability and Bayes' formula we get

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \int p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})\, d\mathbf{z} \tag{4.1}$$

$$\tag{4.2}$$

and

$$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) = p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})p(\mathbf{z}). \tag{4.3}$$

When such models are parameterised using deep neural networks they are termed *deep latent variable models* (DLVM). A benefit of DVLM's is that even when each factor in 4.3 (namely the conditional and prior distributions) are relatively simple (for example, Gaussian), the resulting marginal $p_{\boldsymbol{\theta}}(\mathbf{x})$ can be very complex as a result of 4.1. This is what makes DVLM's attractive when approximating possibly complex distributions.

When attempting to maximise the likelihood of DVLM's we run into the issue that the marginal $p_{\boldsymbol{\theta}}(\mathbf{x})$ is typically intractable due to the integral in 4.1 either not having an analytic solution, or being computationally intractable under estimation. As a consequence we cannot differentiate with regards to (w.r.t) the parameters so as to be able to maximise the likelihood. This intractability also carries over to the posterior distribution $p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})$ via

$$p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}) = \frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})}{p_{\boldsymbol{\theta}}(\mathbf{x})}. \tag{4.4}$$

In order to address these issues of intractability we introduce a parameterised, approximate posterior

$$q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) \approx p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}) \tag{4.5}$$

4.10.4 KL-Divergence

Although there are many methods to penalise the "distance" between two distributions, historically Kullback-Leibler(KL) divergence has been used in the context of VAEs in order to get such a measure of "distance", motivated below in the derivation regarding the evidence lower bound. It must be noted that KL divergence is not a true metric in the sense that it fails to be symmetric.

Formally, KL-divergence is the expected difference in information of one probability distribution, q to another distribution p, where the expectation is taken under q. Mathematically it is defined as

$$D_{KL}(q||p) = \mathbb{E}_{q(\mathbf{x})}\left[\log\left(\frac{q(\mathbf{x})}{p(\mathbf{x})}\right)\right] \tag{4.6}$$

The definition stems from information theory, and can be informally interpreted as how surprised one is from using p as a distribution to describe the process in question when the true distribution of the process is q (contributors 2024).

Recall Jensen's inequality, namely that if $(\Omega, \mathcal{F}, \mu)$ is a measure space, $f : \Omega \to \mathbb{R}$ a measurable function, and $\varphi : \mathbb{R} \to \mathbb{R}$ a convex function, then

$$\varphi\left(\int_\Omega f d\mu\right) \leq \int_\Omega \varphi(f) d\mu. \tag{4.7}$$

If φ is a strictly convex function and f is non-degenerative (i.e.: non-constant) on a set A with $\mu(A) > 0$, then the inequality is strict.

Claim 2. KL divergence is non-negative.

Proof. We have that $-log : \mathbb{R} \to \mathbb{R}$ is a convex function, hence by letting $f = \frac{p(\mathbf{x})}{q(\mathbf{x})}$ we get

$$\begin{aligned} D_{KL}(q||p) &= \mathbb{E}_{q(\mathbf{x})}\left[-\log\left(\frac{p(\mathbf{x})}{q(\mathbf{x})}\right)\right] && (4.8)\\ &\geq -\log\int_{\mathbb{R}} q(\mathbf{x})\frac{p(\mathbf{x})}{q(\mathbf{x})}dx && (4.9)\\ &= -\log\int_{\mathbb{R}} p(\mathbf{x})dx && (4.10)\\ &= -\log(1) = 0 && (4.11) \end{aligned}$$

□

Claim 3. $D_{KL}(q||p) = 0$ if and only if (iff) $p = q \quad \mathbb{P}_{q(\mathbf{x})}$ almost everywhere (a.e).

Proof. ($\Rightarrow$) Suppose that $D_{KL}(q||p) = 0$, then since -log is a strictly convex function, we can only have equality in 4.9 if $-\log\frac{p(\mathbf{x})}{q(\mathbf{x})}$ is degenerative everywhere except on a set of measure zero, ie:

$$\begin{aligned} &D_{KL}(q||p) = 0\\ &\quad\Rightarrow p(\mathbf{x}) = q(\mathbf{x}) \qquad \mathbb{P}_{q(\mathbf{x})} \quad a.e. \end{aligned}$$

($\Leftarrow$)

Suppose that $p = q \quad \mathbb{P}_{q(\mathbf{x})}$ a.e, then

$$\begin{aligned} D_{KL}(q||p) &= \mathbb{E}_{q(\mathbf{x})} \log\left(\frac{q(\mathbf{x})}{p(\mathbf{x})}\right) \\ &= \int_{\Omega} log\left(\frac{q(\mathbf{x})}{p(\mathbf{x})}\right) \mathbb{I}_{\{q(\mathbf{x})=p(\mathbf{x})\}} d\mathbb{P}_{q(\mathbf{x})} \\ &= \int_{\{q(\mathbf{x})=p(\mathbf{x})\}} (\log(1)) d\mathbb{P}_{q(\mathbf{x})} \\ &= 0. \end{aligned}$$

□

4.10.5 Evidence lower bound(ELBO)

As with many problems of estimating the true distribution of a process, $p(\mathbf{x})$, we wish to maximise the likelihood of our data being observed under our approximate, parameterised distribution, $p_{\boldsymbol{\theta}}(\mathbf{x})$. The evidence/variational lower bound provides a lower bound for the log-likelihood of our estimate. Let $q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$ denote our inference model/decoder, $p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})$ be the parameterised but intractable posterior and let $p_{\boldsymbol{\theta}}(\mathbf{x})$ the marginal distribution of $\mathbf{X}$ under our parameterisation.
We have that

$$\begin{aligned} \log p_{\boldsymbol{\theta}}(\mathbf{x}) &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\log p_{\boldsymbol{\theta}}(\mathbf{x})\right] && (4.12) \\ &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\log\left(\frac{p_{\boldsymbol{\theta}}(\mathbf{x},\mathbf{z})}{p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})}\right)\right] && (4.13) \\ &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\log\left(\frac{p_{\boldsymbol{\theta}}(\mathbf{x},\mathbf{z})}{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\frac{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}{p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})}\right)\right] && (4.14) \\ &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\log\left(\frac{p_{\boldsymbol{\theta}}(\mathbf{x},\mathbf{z})}{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\right)\right] + \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\log\left(\frac{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}{p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})}\right)\right] && (4.15) \\ &= \mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathbf{x}) + D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})||p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})) && (4.16) \end{aligned}$$

The first term is called the variational/evidence lower bound (ELBO), denoted $\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathbf{x})$, and the second term is the KL-divergence between $q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$ and $p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})$, denoted $D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})||p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}))$.

Since the KL-divergence is always non-negative, we see that the evidence lower bound is indeed a lower bound for the log-likelihood, i.e.:

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) \geq \mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathbf{x}).$$

Rearranging gives

$$\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathbf{x}) = \log p_{\boldsymbol{\theta}}(\mathbf{x}) - D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})||p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}))$$

which has a very satisfying interpretation. We see that by maximising the ELBO, we not only approximately maximise the marginal log-likelihood of $\mathbf{X}$, but also minimise the KL divergence between the true and approximated posterior distributions.

Manipulating the ELBO we find:

$$\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathbf{x}) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[\log\left(\frac{p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\right)\right] \tag{4.17}$$

$$= \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[\log\left(p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right)\right] - \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[\log\left(\frac{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}{p_{\boldsymbol{\theta}}(\mathbf{z})}\right)\right] \tag{4.18}$$

$$= \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[\log\left(p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right)\right] - D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})||p_{\boldsymbol{\theta}}(\mathbf{z})) \tag{4.19}$$

$$= \text{Reconstruction Loss} - \text{KL Divergence} \tag{4.20}$$

The first term in 4.19 is termed the *reconstruction* loss and the second is the KL divergence between our approximate posterior distribution and our prior distribution.

4.10.6 Deriving the loss function

We now derive an explicit form for the loss function of our neural netowrk, which in our case is the negative ELBO. In order to do this, we must decide on forms for both the prior, posterior and decoder distributions. The most common case, and indeed what we used when implementing the VAE, is to choose the prior and posterior to be Gaussian distributions with diagonal covariance matrices:

$$q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) \sim \mathcal{N}\left(\mathbf{x}, \boldsymbol{\mu}(\mathbf{x}), \text{diag}\left(\boldsymbol{\sigma}^2(\mathbf{x})\right)\right) \tag{4.21}$$

and

$$p(\mathbf{z}) \sim \mathcal{N}(\mathbf{0}, \mathbb{I}) \tag{4.22}$$

where

$$\left(\boldsymbol{\mu}(\mathbf{x}), \log\left(\boldsymbol{\sigma}^2(\mathbf{x})\right)\right) = \text{EncoderNeuralNetwork}_{\boldsymbol{\phi}}(\mathbf{x}) \tag{4.23}$$

Recall that the distribution function for a d-dimensional multivariate gaussian random variable $\mathbf{Y}$ is of the form

$$p_{\mathbf{Y}}(\mathbf{y}) = \left((2\pi)^d|\boldsymbol{\Sigma}|\right)^{-\frac{1}{2}} \exp\left[(\mathbf{y}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{y}-\boldsymbol{\mu})\right] \tag{4.24}$$

where $\boldsymbol{\Sigma}$ is the covariance matrix of $\mathbf{Y}$.

Thus the densities for the prior and posterior are

$$p(\mathbf{z}) = (2\pi)^{-\frac{d}{2}}\exp\left(-\frac{1}{2}\sum_{i=1}^{d}(z_i)^2\right) \tag{4.25}$$

and

$$q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) = \left((2\pi)^d|\boldsymbol{\Sigma}|\right)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}\sum_{i=1}^{d}\left(\frac{z_i-\mu_i}{\sigma_i}\right)^2\right) \tag{4.26}$$

Focusing on the KL Divergence term in the ELBO 4.19 and subbing in the chosen densities for the prior and posterior gives:

$$
\begin{aligned}
&D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})||p_{\boldsymbol{\theta}}(\mathbf{z})) && (4.27)\\
&= \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[\log\left(\frac{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}{p_{\boldsymbol{\theta}}(\mathbf{z})}\right)\right] && (4.28)\\
&= \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[\log q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z})\right] && (4.29)\\
&= \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[-\frac{1}{2}\left(\log(2\pi)^d + \log|\boldsymbol{\Sigma}| - \sum_{i=1}^{d}\left(\frac{z_i-\mu^i}{\sigma_i}\right)^2 + \log(2\pi)^d + \sum_{i=1}^{d}(z_i)^2\right)\right] && (4.30)\\
&= -\frac{1}{2}\left[\log\left(\Pi_{i=1}^{d}\sigma_i\right) + \sum_{i=1}^{d}\frac{\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}(z_i-\mu_i)^2}{\sigma_i^2} - \sum_{i=1}^{d}\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}(z_i^2)\right]. && (4.31)
\end{aligned}
$$

Recalling that $\mathbb{E}(z_i^2) = \sigma_i^2 - \mu_i^2$ and $\text{Var}(z_i) = \mathbb{E}(z_i - \mu_i)^2$, equation 4.31 becomes

$$= -\frac{1}{2}\left[\sum_{i=1}^{d}\left(\log\sigma_i^2 + 1 - \sigma_i^2 - \mu_i^2\right)\right]. \tag{4.32}$$

When dealing with continuous data, the form of the decoder's distribution is assumed to be Gaussian distributed, with mean given by the model's output, and with constant, diagonal covariance structure:

$$p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z}) \sim \mathcal{N}\left(\boldsymbol{\mu}(\mathbf{z}), \boldsymbol{\sigma}^{\mathbf{2}}\mathbb{I}\right) \tag{4.33}$$

where

$$\boldsymbol{\mu}(\mathbf{z}) = \text{DecoderNeuralNetwork}_{\boldsymbol{\theta}}(\mathbf{z})$$

Let k be the dimensionality of the input $\mathbf{x}$, then the reconstruction error term in 4.19 becomes:

$$\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[\log\left(p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right)\right] = \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[-\frac{1}{2\sigma^2}\sum_{i=1}^{k}\left(x_i - \mu(z)_i\right)^2\right) - \frac{k}{2}\log(2\pi\sigma^2)\right] \tag{4.34}$$

Thus the loss function which is the negative ELBO becomes:

$$Loss = \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[\frac{1}{2\sigma^2}\sum_{i=1}^{k}\left(x_i - \mu(z)_i\right)^2\right) - \frac{k}{2}\log(2\pi\sigma^2)\right] - \frac{1}{2}\left[\sum_{i=1}^{d}\left(\log\sigma_i^2 + 1 - \sigma_i^2 - \mu_i^2\right)\right] \tag{4.35}$$

4.10.7 Gradient Estimates

If we wish to use gradient based optimisation techniques such as Stochastic Gradient Descent (SGD) in order to maximise the ELBO (or equivalently, minimise the loss function), we need to be able to find unbiased estimators for the gradient of the ELBO w.r.t both sets of parameters $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$, i.e. we need to get unbiased estimates for:

$$
\begin{aligned}
\nabla_{\boldsymbol{\theta}\boldsymbol{\phi}}\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathbf{x}) &= \nabla_{\boldsymbol{\theta}\boldsymbol{\phi}}\left(\text{Reconstruction Loss} - \text{KL divergence}\right) && (4.36)\\
&= \nabla_{\boldsymbol{\theta}\boldsymbol{\phi}}\left(\text{Reconstruction Loss}\right) - \nabla_{\boldsymbol{\theta}\boldsymbol{\phi}}\left(\text{KL divergence}\right) && (4.37)\\
& && (4.38)
\end{aligned}
$$

Finding gradients for the KL divergence loss term is straightforward as the loss depends only on the outputs of the encoder, which in our case is a differentiable function.
Let us now attempt to find the gradient of the Reconstruction Loss.

We first consider taking the gradient w.r.t $\boldsymbol{\theta}$:

$$\begin{aligned}\nabla_{\boldsymbol{\theta}} \left(\text{Reconstruction Loss}\right) &= \nabla_{\boldsymbol{\theta}} \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\log\left(p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right)\right] && (4.39)\\ &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\nabla_{\boldsymbol{\theta}} \log\left(p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right)\right] && (4.40)\\ &\eqsim \nabla_{\boldsymbol{\theta}} \log\left(p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right) && (4.41)\end{aligned}$$

where 4.41 is an unbiased Monte Carlo estimate of 4.40, gotten by sampling from $\mathbf{z} \sim q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$.

We however run into issues when attempting to take the derivative w.r.t $\boldsymbol{\phi}$ as we can no longer simply move the derivative inside the integral due to us taking the integral relative to $q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$ which depends on $\boldsymbol{\phi}$. A work around is if we can somehow express the stochastic element of $q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$ by something independent of $\boldsymbol{\phi}$, allowing us to differentiate through expectation after a change of variables. This approach is famously known as the reparameterisation trick.

4.10.8 The Reparameterisation Trick

Suppose we can express the random variable $\mathbf{z} \sim q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$ as the transformation of another random variable $\boldsymbol{\epsilon}$, ie:

$$\mathbf{z} = g(\boldsymbol{\phi}, \mathbf{x}, \boldsymbol{\epsilon}) \tag{4.42}$$

where $\boldsymbol{\epsilon}$ has some distribution independent of $\boldsymbol{\phi}$ and $\mathbf{x}$, say $\boldsymbol{\epsilon} \sim p(\boldsymbol{\epsilon})$, and g is an invertible and differentiable function.

We can now express expectations under $q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$ as being under $p(\boldsymbol{\epsilon})$:

$$\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} f(\mathbf{z}) = \mathbb{E}_{p(\boldsymbol{\epsilon})} f(\mathbf{z}) \tag{4.43}$$

and because we are now taking expectation under a distribution independent of $\boldsymbol{\phi}$, the derivative and expecations become commutative as was the case with $\boldsymbol{\theta}$:

$$\begin{aligned}\nabla_{\boldsymbol{\phi}} \left(\text{Reconstruction Loss}\right) &= \nabla_{\boldsymbol{\phi}} \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\log\left(p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right)\right] && (4.44)\\ &= \nabla_{\boldsymbol{\phi}} \mathbb{E}_{p(\boldsymbol{\epsilon})} \left[\log\left(p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right)\right] && (4.45)\\ &= \mathbb{E}_{p(\boldsymbol{\epsilon})} \left[\nabla_{\boldsymbol{\theta}} \log\left(p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right)\right] && (4.46)\\ &\eqsim \nabla_{\boldsymbol{\theta}} \log\left(p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right) && (4.47)\end{aligned}$$

where line 4.47 is again a Monte Carlo estimator of 4.46, however samples of $\mathbf{z}$ are now gotten by randomly sampling $\boldsymbol{\epsilon}$ and applying the transformation $\mathbf{z} = g(\boldsymbol{\phi}, \mathbf{x}, \boldsymbol{\epsilon})$.

We are thus now able to take the derivates of the ELBO w.r.t both the encoder and decoder's parameters, allowing us to use gradient based optimisation techniques to maximise the ELBO.

4.10.9 Final Loss

Revisiting the Reconstruction Loss in 4.34 and using our new reparameterisation trick to consider the derivative within the expectation, we see that when taking the derivatives w.r.t to $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$, the second term falls away (as it is independent of both the encoder and decoder's parameters), and the constants in front of the summation can be disregarded, meaning we just need to consider terms of the form:

$$-\sum_{i=1}^{n}\left(x_i - \mu(z)_i\right)^2) \tag{4.48}$$

which is the negative of the familiar Mean Squared Error Loss (MSE).

Thus, by choosing the loss to be the negative ELBO, and using the fact that we will be taking unbiased estimators of the gradient of this loss via sampling under $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbb{I})$, we arrive at the final loss function for the j_{th} sample $\mathbf{x}_j$:

$$Loss_j = \sum_{i=1}^{k}\left(x_{ij} - \mu(z)_{ij}\right)^2) - \frac{1}{2}\sum_{i=1}^{d}\left(\log\sigma_{ij}^2 + 1 - \sigma_{ij}^2 - \mu_i^2\right) \tag{4.49}$$

$$= \text{MSE}_{\text{loss}} + \text{KL}_{\text{loss}} \tag{4.50}$$

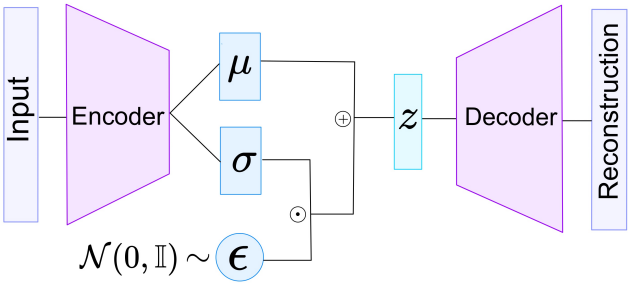


Figure 2: Variational Auto-Encoder Architecture.

4.10.10 Conditional VAEs

A modification of the usual VAE architecture was used in this implementation, known as the Conditional VAE (CVAE). A slight modification allows one to append another input to be conditioned on for both the encoder and decoder networks. In our case this has particular relevance in allowing us to condition on the most recent window of time when predicting the next signature, as intuitively the most recent events in time have the most impact on current events. By allowing both the encoder and decoder to take in these conditional inputs, we are also able to condition upon the previously generated signature during market simulation, meaning that generated paths will be more coherent.

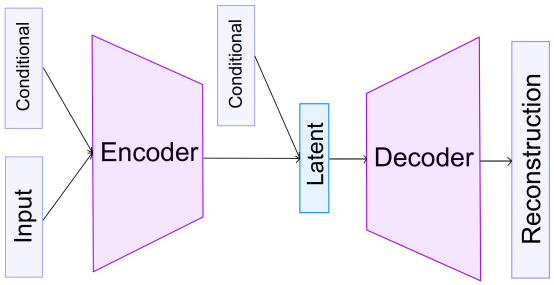


Figure 3: Conditional Variational Auto-Encoder Architecture.

4.11 Recurrent Neural Networks (RNNs)

Recurrent Neural Networks are a class of neural network architecture that were created to handle sequential data, where "memory" of previous inputs would help in the predictive power of the model. A naive approach is to simply put as input to a standard neural network all historical data that one might want the model to consider when making a prediction, however this approach has many drawbacks. Namely, it cannot handle historical data of varying length, and there is no inherent way for the neural network to understand the order in which the data appears (although a simple time index appendage to each data-point is a possible workaround for this). RNNs help address these issues.

4.11.1 Vanilla RNNs

Vanilla RNNs hold a memory of past inputs by maintaining a collection of hidden states (one for each layer). At time t, the hidden state of layer 1, $\mathbf{h}_t^1$ is calculated as a function of the previous time step's hidden state at layer 1, $\mathbf{h}_{t-1}^1$, alongside the latest input vector being considered in the sequence, $\mathbf{x}_t$. At any other layer l, the hidden states $\mathbf{h}_t^l$ is similarly defined as a function of $\mathbf{h}_{t-1}^l$ and $\mathbf{h}_t^{l-1}$. The final output of the hidden layer is then a linear transformation of the final hidden

state. More specifically, for a network with L hidden layers we have that:

$$\mathbf{h}_t^1 = \tanh\left(\mathbf{W_h}^1 \mathbf{h}_{t-1}^1 + \mathbf{W_x}^l \mathbf{x}_t + \mathbf{b}^1\right) \tag{4.1}$$

$$\mathbf{h}_t^l = \tanh\left(\mathbf{W_h}^l \mathbf{h}_{t-1}^l + \mathbf{W_x}^l \mathbf{h}_t^{l-1} + \mathbf{b}^l\right) \tag{4.2}$$

$$\mathbf{output}_t = \sigma\left(\mathbf{W}_o \mathbf{h}_t^L + \mathbf{b}_o\right) \tag{4.3}$$

The major drawback of RNNs is that they suffer from what is know as the issue of vanishing (or exploding) gradients. To explain how this issue arises let's consider some input in our sequence of inputs at time s, $\mathbf{x}_s$. The only way for information about $\mathbf{x}_s$ to propogate forward in time is through our hidden states $\mathbf{h}^l$, however, at each new point in our sequence, the previous hidden state is multiplied by the weight matrix $\mathbf{W_h}^l$. Due to regularisation and initialisation, this weight matrix likely has small values in the range $(-1, 1)$. What this means is that if we are now at time t with $s << t$, then the effect of $\mathbf{x}_s$ on our current hidden state $\mathbf{h}_t^l$ (and consequently our output) would become vanishingly small due to successive multiplications by $\mathbf{W_h}^l$. Due to this tendency for inputs in the distant past to have a negligible effect on our current output, Vanilla RNNs are said to have short-term memory.

4.11.2 Long Short-Term Memory (LSTM) Models

LSTM models are a form of RNN which were created in order to address the short-term memory issues of Vanilla RNNs, attempting to additionally give them long term-memory.

The LSTM differs from standard RNNs in that it has both a cell and hidden state at each layer. The cell state $\mathbf{C}$ helps store and propogate long-term information about sequential inputs, whilst the hidden state $\mathbf{h}$ (as in Vanilla RNNs) can be viewed as a short term memory mechanism.

The architecture includes a *forget gate* $\mathbf{f}$, an *input gate* $\mathbf{i}$, an *output gate* $\mathbf{o}$ and a candidate cell state $\tilde{\mathbf{C}}$. The forget gate decides how much of the previous time step's cell state to remember. The input gate governs how much of the candidate cell state will be included in the current cell state. The output gate determines which parts of the the cell state are used for the hidden state (and consequently the output). The candidate cell state governs which new information to incorporate into the long-term memory. More formally, we have that

$$\mathbf{f}_t = \sigma\left(\mathbf{W}_{fh}\mathbf{h}_{t-1} + \mathbf{W}_{fx}\mathbf{x}_t + \mathbf{b}_f\right) \tag{4.4}$$

$$\mathbf{i}_t = \sigma\left(\mathbf{W}_{ih}\mathbf{h}_{t-1} + \mathbf{W}_{ix}\mathbf{x}_t + \mathbf{b}_i\right) \tag{4.5}$$

$$\tilde{\mathbf{C}}_t = \tanh\left(\mathbf{W}_{Ch}\mathbf{h}_{t-1} + \mathbf{W}_{Cx}\mathbf{x}_t + \mathbf{b}_C\right) \tag{4.6}$$

i.e. the forget gate, input gate and candidate cell state are all simply functions of the previous hidden state, and the current input.

The cell state, output gate and hidden state are defined as follows:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_\mathbf{t} \tag{4.7}$$

$$\mathbf{o}_t = \sigma\left(\mathbf{W}_{oh}\mathbf{h}_{t-1} + \mathbf{W}_{ox}\mathbf{x}_t + \mathbf{b}_o\right) \tag{4.8}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \tag{4.9}$$

$$\mathbf{output}_t = \sigma\left(\mathbf{W}\mathbf{h}_t + \mathbf{b}\right) \tag{4.10}$$

where $\odot$ denotes the element-wise multiplication of two vectors.

These definitions extent naturally to the multilayer case by simply replacing all $\mathbf{x}_t$'s with the previous layer's hidden state output when not considering the very first layer, as was the case for the Vanilla RNN. For example:

$$\mathbf{f}_t^l = \sigma\left(\mathbf{W}_{fh}^l \mathbf{h}_{t-1}^l + \mathbf{W}_{fx}^l \mathbf{h}_t^{l-1} + \mathbf{b}_f^l\right) \tag{4.11}$$

We see that the cell state is an element-wise combination of the previous cell state and the new candidate cell state. In this way, we have removed the repeated multiplication by a weight matrix, meaning that long term information does not necessarily get degraded over time.

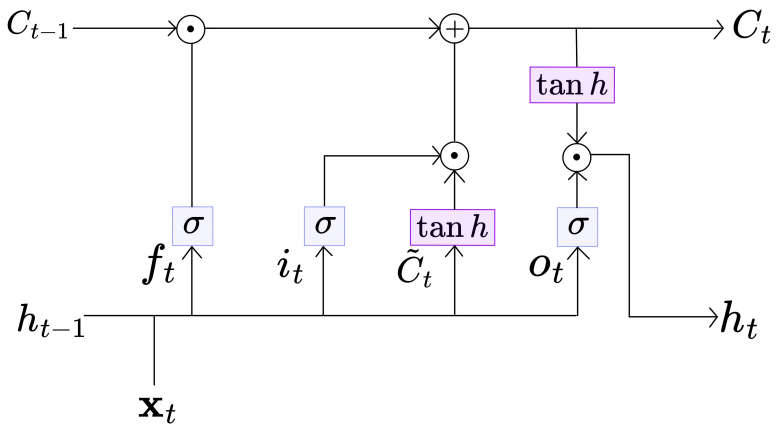


Figure 4: single-layer LSTM Architecture

4.12 Deep Hedging

With the proliferation of the use of DNNs to tackle previously unsolved problems, the issue of whether or not DNNs can be effectively used for the problem of hedging financial instruments has been explored greatly. Thus, it stands that the research questions asked in this paper are not wholly original, although as far as we are aware no one has ever done this specific implementation of joint-distribution generation via path signatures to train a best-of basket option hedger.

The paper of Buehler, Gonon, et al. 2019 showed that recurrent neural networks can effectively reproduce the hedging results of more traditional models when no transaction costs are considered, and could in fact provide better performance when transaction costs are included.

The work of Stangroom 2023 expanded upon these findings, showing that the more specific LSTM architecture further improves hedging capabilities, and that the use of pseudo-real path generation offers a performance boost to DNN hedgers.

5 Methodology

5.0.1 Reproducibility

Throughout all of our code and for all libraries which use pseudo-random number generation, we fixed our seeds to the value of 2024 should the reader like to reproduce our results.

All the code written for the project can be found in the GitHub repository https://github.com/NickBossi/Hedging-Honours-Project (Bossi 2024).

5.1 Path Generation

5.1.1 Data Collection

The two historical stock prices of Apple (Yahoo Finance n.d.) and Microsoft (Finance n.d.) were downloaed from the Yahoo! finance website and the Opening price of the stocks for the period of 26/06/2020-17/06/2024, corresponding to 1000 days of historical price data.

5.1.2 Data Pre-Processing

The 1000 days of historical price data was divided into 200 subpaths of dimension 2 (for the two stocks), each of length 5 days. The log-signature of each of these 200 (5 x 2) subpaths were then taken using the iisignature package Graham and Reizenstein 2024, each producing a log-signature of dimension 14. After this, the data was normalised to be in the interval $[0, 1]$ using sklearn MinMaxScaler, which linearly scales down features to be in a fixed range, thus not reducing the effect of outliers. The parameters for the scaler were stored so that they could eventually be used to denormalise the generated samples. These normalised log-signatures were then used as input to train the CVAE.

5.1.3 CVAE

The hyperparameters for the CVAE we informed by those used in Buehler, Horvath, Lyons, Arribas, et al. 2020, with adjustments being made to both the KL weighting and number of nodes in the hidden layers in order to accommodate the difference in data (the simulation of two paths, instead of one). The CVAE followed a simple design with three hidden layers: one for the encoder, one for the latent space and one for decoder.
The table below summarises the hyperparameters used:

Hyperparameter	**Value**
Kl weighting	0.003
latent dimension	8
width of hidden layers	100
hidden layer activations	LeakyReLU
LeakyReLU negative slope	0.3
output activation	sigmoid
batch size	16
learning rate	0.0003
number of epochs	20 000

Table 1: Hyperparameter Configuration of VAE

5.1.4 Loss function

With the pre-determined KL weighting of 0.003, the loss function for the VAE per training example becomes

$$Loss = 0.997 \times \text{MSE} + 0.003 \times \text{KL}_{\text{loss}}$$

where the MSE and KL_{loss} terms are calculated as in 4.49.

5.1.5 Generation

After training, the marginal distributions of the latent embeddings of the training data were plotted, visualised 5 below:

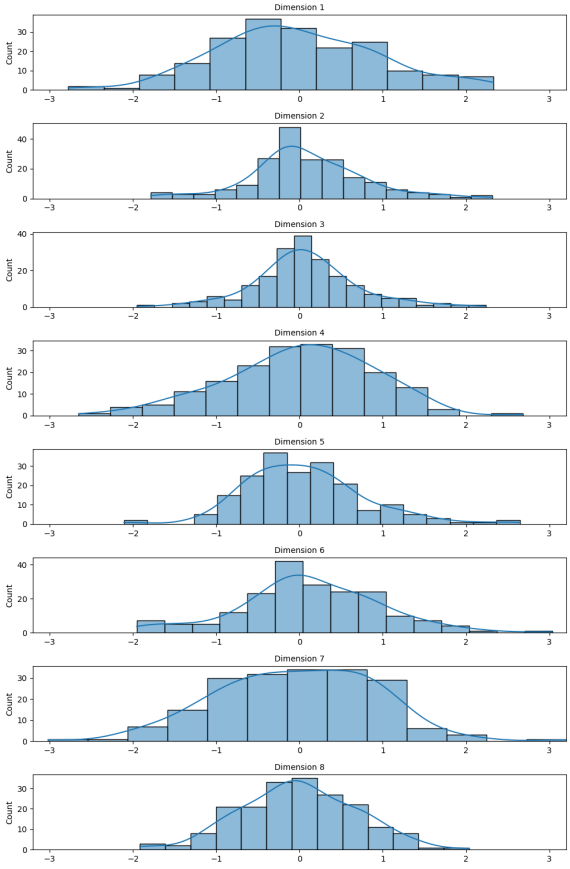


Figure 5: Box plots of latent dimensions.

We note that although the shape of each latent dimension is approximately standard normal, the exact mean and variance varies between dimensions. This is because we lowered the weighting of the KL divergence term as otherwise the model was over-regularised, causing latent space collapse (i.e. all of the generated samples looked the same).

Standard practice in VAE's is to simply sample from the prior distribution $z \sim \mathcal{N}(\mathbf{0}, \mathbb{I})$ in order to generate novel samples, however, in light of the above latent dimension projections, we decided to sample from a multivariate gaussian $z \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ where $\boldsymbol{\mu}_i$ is simply the mean of the i^{th} dimension's latent embeddings, and $\boldsymbol{\Sigma}$ is a diagonal covariance matrix where $\boldsymbol{\Sigma}_{ii}$ is the variance of the i^{th} dimension's latent embeddings.

After the CVAE was trained, 100 000, 30 day paths were simulated. This was done by using the CVAE to generate 6 5-day log-signatures, each conditioned on the previous time-period's log-signature. The pseudo-code below describes the exact methodology followed:

Algorithm 1 Log-signature Generation

```
log_signature_paths ← []
for i = 1 to 100,000 do
    conditional ← log-signature of the last path in training data
    path ← []
    for j = 1 to 6 do
        Sample z ~ N(μ, Σ)
        x ← CVAE_Decoder(z, conditional)
        conditional ← x
        Append x to path
    end for
    Append path to log_signature_paths
end for
```

5.1.6 Inversion

The iisignature package Graham and Reizenstein 2024 was then used to exponentiate each log-signature into a signature, and then the Signatory package Kidger and Lyons 2021 was used to convert these signatures into time-series paths. It must be noted that the inversion of path signatures back into real-world paths is highly nontrivial, and the Signatory package is limited to returning paths of length = depth + 1, where depth is the depth of the signature produced. This was in fact why we generated paths of length 5, as taking signatures of higher depths was computationally too expensive.

The full algorithm for the path inversion is as follows:

Algorithm 2 Signature Inversion

```
paths ← []
for i = 1 to 100,000 do
  path ← []
  for j = 1 to 6 do
    signature ← logsig_to_sig(log_signature_paths[i][j])
    6_day_path ← signature_inversion(signature)
    if i > 1 then
      Shift 6_day_path to have its first value equal the previous 6_day_path's final value
    end if
    5_day_path ← 6_day_path[1:]
    Append 5_day_path to path
  end for
end for
```

5.2 Deep Hedging

5.2.1 Data

The data used to train the Deep Hedger is of course the paths generated using the CVAE. 100 000 paths were simulated and this data was randomly split into training, validation and testing datasets using a 70/20/10 split.

5.2.2 Feed-Forward Neural Network Architecture

The simple feed-forward neural network takes in as input the two stock's current prices and the time to maturation of the option, ie:

$$\text{input} = \left[S_t^1, S_t^2, T - t\right]$$

and outputs ϕ, the holdings of the two stocks in the portfolio at time t, ie:

$$\text{output} = \boldsymbol{\phi}_t = \left[\phi_t^1, \phi_t^2\right].$$

5.2.3 LSTM Architecture

As described, the LSTM takes in as input a sequence of inputs. In our case this is the sequence of stock prices associated with our period of training alongside the time to maturation of the option, ie:

$$\begin{aligned}
&\text{input} = \mathbf{x} \\
&\mathbf{x} = [\mathbf{x}_0, ..., \mathbf{x}_{T-1}] \\
&\mathbf{x}_t = \left[S_t^1, S_t^2, T - t\right]
\end{aligned}$$

Note that the last day of stock data (i.e. the day of maturation) does not get included as an input, as we can only make trades on days leading up to but not including the day of maturation of the option.

5.2.4 Loss function

Prior to training, the option prices $H_i = [H_0, H_{i,1}, H_{i,2}, ..., H_{i,T}]$ were calculated for each input path $1 \leq i \leq 100\ 000$ as informed by the Black-Scholes equation as described in 4.7.30, where $H_{i,T} = 100 \times \max\left\{\frac{S_T^1}{S_0^1}, \frac{S_T^2}{S_0^2}\right\}$ is the actual payoff of the i^{th} path. Note that the option price H_0 is the same for all paths, as this is the price the actual option is sold at at time $t = 0$ and is thus path independent.

For each input path, the loss was calculated to be the sum of squared error between the changes in the value of our portfolio and the changes in the option's price, namely:

$$loss_i = \sum_{t=1}^{T} \left[\Delta V_{i,t} - \Delta H_{i,t}\right]^2 \tag{5.1}$$

$$= \sum_{t=1}^{T} \left[(V_{i,t} - V_{i,t-1}) - (H_{i,t} - H_{i,t-1})\right]^2 \tag{5.2}$$

$$\tag{5.3}$$

where

$$V_{i,t} = V_{i,0} + \sum_{j=1}^{t} (\Delta \boldsymbol{S} \cdot \boldsymbol{\phi}_j) \tag{5.4}$$

$$= H_0 + \sum_{j=1}^{t} \left[\left(S_j^1 - S_{j-1}^1\right)\phi_{j,1} + \left(S_j^2 - S_{j-1}^2\right)\phi_{j,2}\right] \tag{5.5}$$

is the value of our portfolio at time t as governed by the trading strategy given by our neural network.

This choice of loss function is naturally motivated by the fact that if the changes in our portfolio's value closely tracks that of the changes in the true option's value, then the final value's should be similar.

5.2.5 Hyperparameter Search

For both the Feed-Forward and LSTM models, a hyperparameter search was conducted using the Optuna library Akiba et al. 2019, using the validation loss on the 20 000 validation set as a means of determining optimal hyperparmeters.

Optuna uses a median criterion for pruning unpromising runs. More specifically, if the validation loss of a run after epoch k is more than the median loss of all previous runs after epoch k then that run will be stopped prematurely so that more promising runs can be explored. A possible issue that arises from this is that different runs have different learning rates, and so a certain run might be doing worse near the beginning of training but then eventually end up outperforming another run later on. We also have the issue that if somehow the first few runs performed relatively well, then many future runs will be culled off which might limit proper exploration of the hyperparameter space. In order to try combat these shortcomings, we only allowed pruning to take place after 10 full runs had occurred, and thereafter only after 10 epochs of training (per run).

5.2.6 Feed-Forward Hyperparameters

We decided to do 500 runs each of 100 epochs in order to effectively explore the relatively large search space in a reasonable amount of time given our computational and time constraints. The hyperparameters, their ranges, as well as the optimal values found are presented in the table below. Note that continous ranges are depicted with [ ], whilst finite, categorical ranges are depicted with {}.

Hyperparameter	**Range/Values Considered**	**Optimal Value Found**
activation	{ReLU, Tanh, LeakyReLU}	LeakyReLU
learning rate	[1e-5, 1e-2]	5.377e-3
batch size	{64, 256, 1024, 4096, 8192}	8192
optimizer	{Adam, SGD}	Adam
l1 regularisation	[0, 1e-2]	3.643e-7
l2 regularisation	[0, 1e-2]	7.399e-3
number of layers	[1,5]	1
width layer 1	[32, 512]	169
dropout layer 1	[0, 0.3]	2.371e-1

Table 2: Hyperparameter search space and optimal hyperparameters found for Feed-Forward Hedger

We note that all optimal values found fall reasonably within the ranges considered, except that of the batch size, whose optimal value was the upper bound of the range considered. When the batch size was increased beyond 8192 we had errors with GPU memory allocation, so this is the size we settled on.

5.2.7 LSTM Hyperparameters

Before conducting a full hyperparameter search for the LSTM we trained a non-tuned LSTM as a sanity check to see if there was any potential benefit in using a LSTM architecture over a standard feed-forward one. This model will henceforth be referred to as the Untuned LSTM model. The (randomly chosen) hyperparameters for this model are captured in the table below:

Hyperparameter	**Value**
learning rate	0.001
batch size	2048
optimizer	Adam
l1 regularisation	0
l2 regularisation	0
number of layers	2
width layer 1	128
width layer 2	128
dropouts	0

Table 3: Hyperparameters for Basic LSTM

Due to the increased complexity and consequently training time of the LSTM vs standard feed-forward models, we had to reduce the number of runs and epochs over which the hyperparameter

tuning was done. Thus the LSTM hyperparameter tuning took place for 100 runs each over 50 epochs.

Hyperparameter	**Range/Values Considered**	**Optimal Value Found**
learning rate	[1e-5, 1e-2]	1.334e-4
batch size	{64, 256, 1024}	1024
optimizer	{Adam, SGD}	Adam
l1 regularisation	[0, 1e-2]	3.690e-5
l2 regularisation	[0, 1e-2]	3.380e-3
number of layers	[1,5]	1
width layer 1	[32, 512]	432
dropout layer 1	[0, 0.3]	9.033e-1

Table 4: Hyperparameter search space and optimal hyperparameters found for LSTM Hedger

5.2.8 Final Training

The Feed-Forward, Tuned LSTM and Untuned LSTM models were all then trained for 1000 epochs on the full 90 000 non-test pseudo-generated paths using the hyperparameters as found above.

5.2.9 Testing Hedging Performance

Our hypothesis is that the Deep Hedging Models will outperform Black-Scholes in the hedging of financial instruments. In order to test this hypothesis we need to use both Black-Scholes and our trained models to make trades in order to hedge against the option of interest. To this end we have two sets of test results. Firstly we use all the models to trade on the 10 000 pseudo-generated test paths and compare the profit and losses (PNLs) across all these paths. The PNL for a single path is calculate as:

$$\text{PNL} = V_T - H_T \tag{5.6}$$

i.e. it is the difference of the final value of our portfolio under our trading strategy vs the final value of the option.

For the Black-Scholes model, the hedging strategy used is simply that as informed by 4.7.30. i.e. At every time t, we simply have our holdings for stock 1 and stock 2 being

$$\text{holdings}_t = \left[\Phi\left(d_1^{\frac{i}{j}}(t)\right), \Phi\left(-d_2^{\frac{i}{j}}(t)\right)\right]. \tag{5.7}$$

For all the neural networks, our holdings at time t are simply the output of the neural network given the input up to time t,

$$\text{holdings}_t = \text{model}(\text{input}_t). \tag{5.8}$$

The very final test of the models is how each performs on the true real-world 30 day stock path. Since this would only result in a single PNL, to get a better idea of how well each model tracked

the price of the option we additionally calculated the Mean-Squared-Error between the changes in each model's portfolio value, and the changes in the actual option's price:

$$\text{MSE} = \frac{1}{30} \sum_{t=1}^{T} [\Delta V_t - \Delta H_t]^2 \tag{5.9}$$

6 Results

6.1 Path Generation

6.1.1 Training Loss

The plots below shows the Mean Squared Error (MSE) loss, Total loss and KL koss of the VAE over the 20 000 epochs of training. Please note that the MSE and Total losses were plotted on a log-scale, whilst the KL loss was plotted on a linear scale.

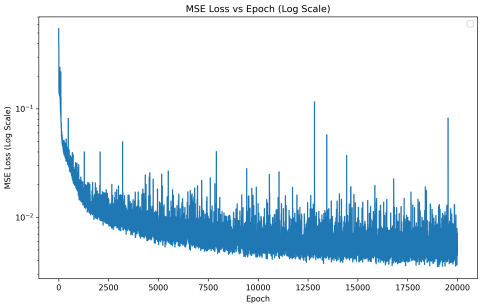


Figure 6: VAE MSE loss

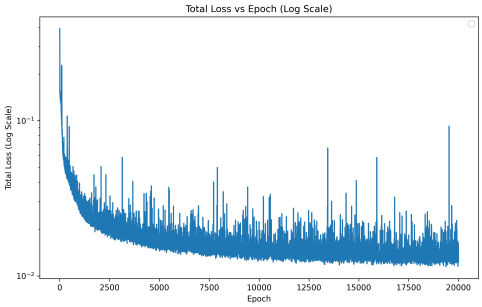


Figure 7: VAE Total loss

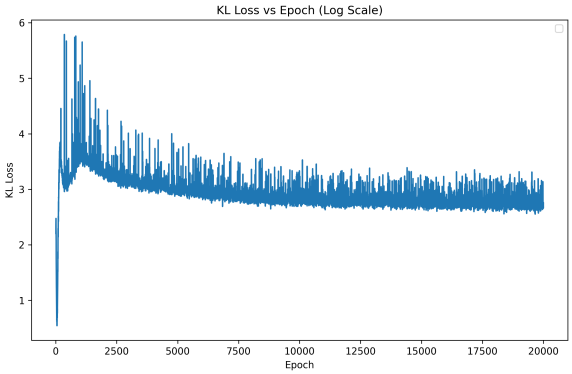


Figure 8: VAE KL loss

We note that the MSE loss (and consequently the total loss) quickly saturates, and the only thing really changing during later epochs is the slow decrease in KL divergence loss as the neural network attempts to coerce the latent space's distribution to more closely resemble that of a standard Gaussian.

6.1.2 Distribution Similarity

The effectiveness of our hedger depends upon the ability of our VAE to generate samples from the same distribution as the true sample path.

As a sanity check, we went through the historical stock data used to train the VAE, and then used the VAE to generate each 5-day window, conditioned on the previous real 5-day window in the training data. The real historical paths and our new generated paths were then plotted against each other:

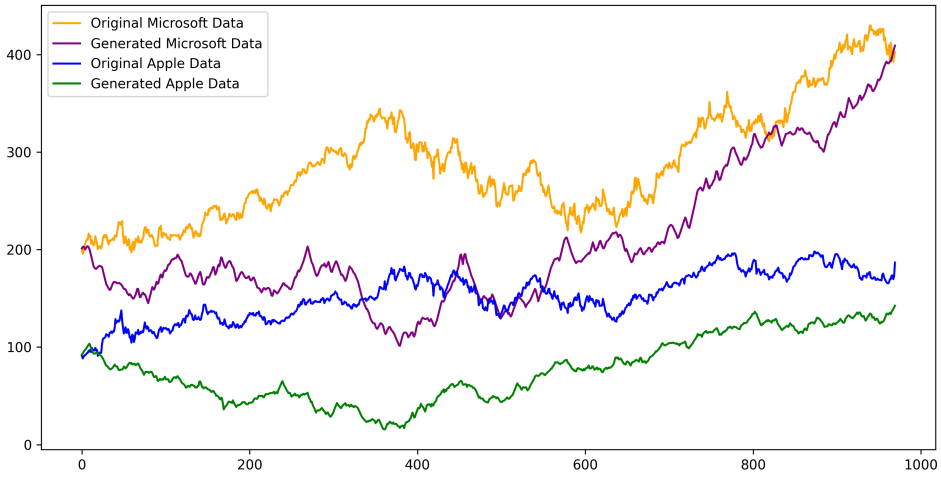


Figure 9: Original and Generated Stock paths over 1000 day historical period

We see that both generated paths end up reasonably close to the final prices of the true paths. More importantly, our generation effectively captures qualities of the joint distribution of the stocks, such as correlation and covariance, which can be seen by the fact that similar movements happen in each stock over similar periods of time. This similarity in movement is exactly what is seen in the original stocks, and is what would be expected of two stocks representing companies from the same industry (in this case the tech industry).

6.1.3 Distribution of Gains

In the context of Hedging, we are mainly concerned with ensuring that the distribution captured by the VAE is representative of the true gains and losses of the real-world data. This is especially true in our particular option's case, where the value of the option is a direct consequence of the movements of the two stocks relative to their initial price. Again we used the 200 5-day generated samples as above, but this time compared the generated movements (i.e. final value - initial value) of our generated samples with those of the real-world 5-day movements. This comparison is captured in the plots below:

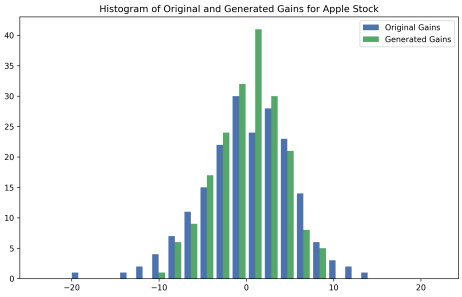


Figure 10: Apple Gains

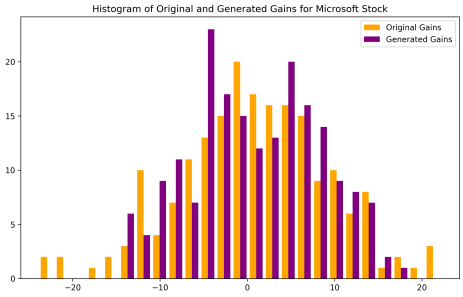


Figure 11: Microsoft Gains

We see that the gains for each stock are similar, although the original movements for both stocks is more spread out, which might have negative implications for the robustness of our hedger.

6.1.4 Kolmogorov-Smirnov Test

The two-sample Kolmogorov-Smirnov (K-S) test tests if two empirical distributions are different to one another. The theory motivating the effectiveness of the test can be found in Mathematics 2024, and a practical implementation of the test can be found in Bolen et al. 2024. The K-S test is a non-parametric test which compares the CDF's of the two distributions in question by finding the maximal difference in their CDF's over their support. More formally, the K-S test statistic is

$$D_{n,m} = \sup_x |F_{1,n}(x) - F_{2,m}(x)| \tag{6.1}$$

where $F_{1,n}(x)$ and $F_{2,m}(x)$ are the empirical CDF's of the two distributions, and n and m the number of samples from each. For large samples, the null hypothesis is rejected at level $p = 0.05$ when

$$D_{n,m} \geq 1.358\sqrt{\frac{n+m}{nm}}. \tag{6.2}$$

The null hypothesis for the test is that the distributions are the same, i.e., when comparing two random variables X and Y, we have that

$$H_0 : F_X = F_Y$$
$$H_1 : F_X \neq F_Y.$$

We performed K-S tests between the generated and original gains for both of the stocks. The results the two tests are captured in the table below:

	KS Test-Statistic	**p-value**
Apple Stock	0.1031	0.2545
Microsoft Stock	0.0515	0.9596

Table 5: p-values from two K-S tests for Apple and Microsoft stocks

We see that neither p-values are near the threshold of 0.05, and thus we cannot say that the samples

come from different distributions.

Although this test does not "prove" that the two distributions are the same, it at least tells us that there is not enough evidence to suggest that the distributions are different.

6.1.5 Conditional Generation

A major proponent of the generation is that we are attempting to sample from a conditional distribution rather than an unconditional one. In order to test whether or not this conditioning was effective, we used the true and generated 5 day windows over the period of the training data. We shifted all 5 day windows to begin and zero, and then calculated the MSE between the generated 5 day window to that of the true 5 day window. We then randomly chose another real-world 5 day window from the training data, also shifted down to zero, and calculated the MSE between this randomly chosen path and the true path. Finally we took the mean of the real vs generated MSEs and the mean of the real vs random MSEs. The table below compares the MSEs for both path 1 and path 2, and then the final overall MSEs across both paths:

	Real vs Generated	**Real vs Random**
Path 1 MSE	17.7778	24.2246
Path 2 MSE	56.3932	76.4926
Total MSE	37.0855	50.3586

Table 6: MSEs of Real vs Generated and Real vs Random subpaths

We see that on average the conditionally generated paths are significantly closer to the real paths than the real paths to a randomly selected path. This indicates that our VAE has effectively learnt the conditional distribution.

6.1.6 Generating Future Samples

Our main interest is simulating both stocks 30 days into the future, as this is the period over which we will be trading in an attempt to replicate the option value.

The plots on the next page show 1000 generated samples for both the Apple and Microsoft stock prices over this 30 day period.

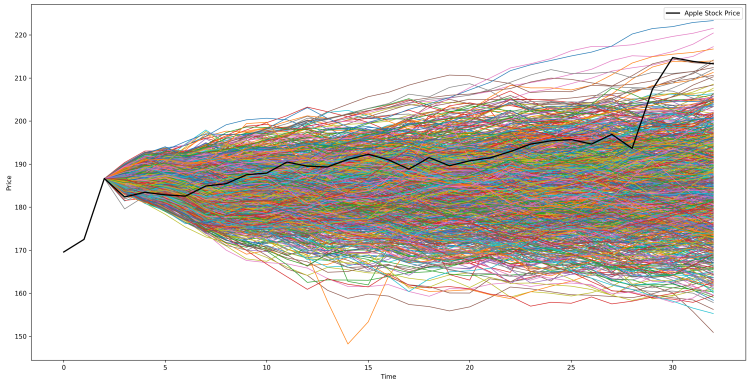


Figure 12: Original and Generated Apple Stock paths

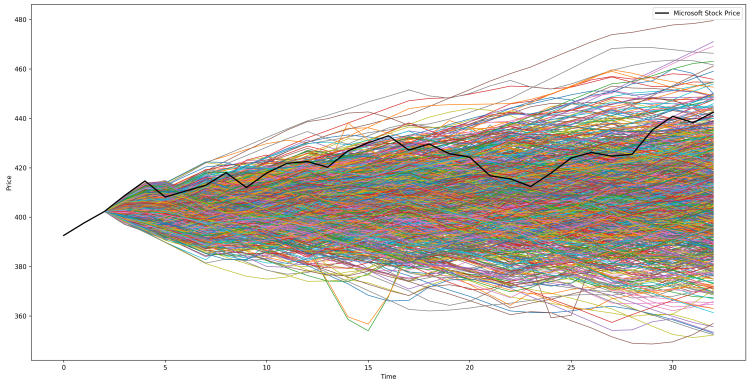


Figure 13: Original and Generated Microsoft Stock paths

We note that the generated paths for both stocks have a good spread whose range appears to be realistic when compared to the real-world data in the 30 day generated period.

6.2 Deep Hedging

6.2.1 Feed-Forward Loss curves

For the Feed-Forward network, we recorded both the training and test set loss at each epoch of training, depicted in the log-scaled plot below:

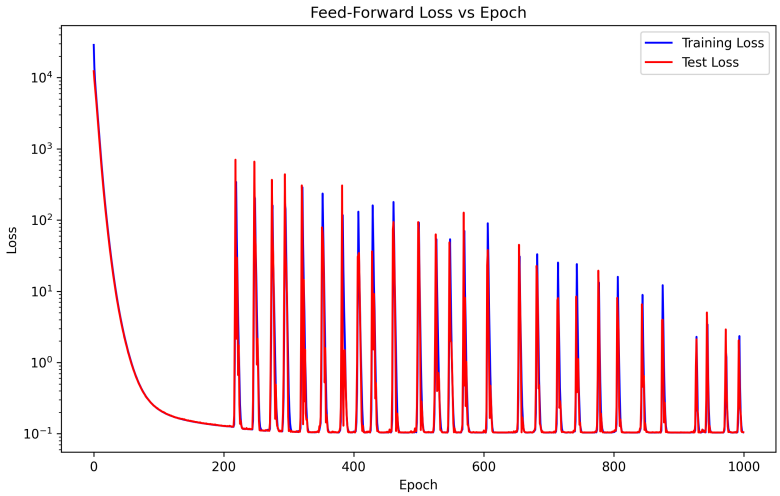


Figure 14: Feed-Forward Training and Test Loss Curves

We see that there are clear spikes in loss which gradually decrease over time. There are many possible explanations to this, one of which is the way in which updates to the model's parameters occur in batches. More specifically, within an epoch we update the model k times, where k is the number of batches in our training set. The loss used to make an update to the parameters of the model is the mean loss of a single batch. Now, due to the random allocation of training samples to batches, if many of the samples in a single batch happen to be outliers in the data or samples which the model struggles to fit, the loss incurred by this batch will be very large when compared to the other batches and consequently the effect on the change in the model parameters will be large (since the size of update is proportional to the loss incurred). This means the model will likely over-correct to try and accommodate these samples, meaning that in the next batch or epoch the loss could spike as the model is much worse at predicting the output on average. Due to the use of a momentum optimiser such as Adam, the over-correcting nature of these updates decreases over time, accounting for the fact that the peaks of these spikes decreases.

We also note that the test losses decrease almost identically to the training, which helps give us confidence that the hyperparameters found do indeed help the model generalise to unseen data.

6.2.2 Tuned LSTM Loss Curves

Similarly, we plotted the training and test losses for the Tuned LSTM, but this time on a linear scale due the comparatively small range.

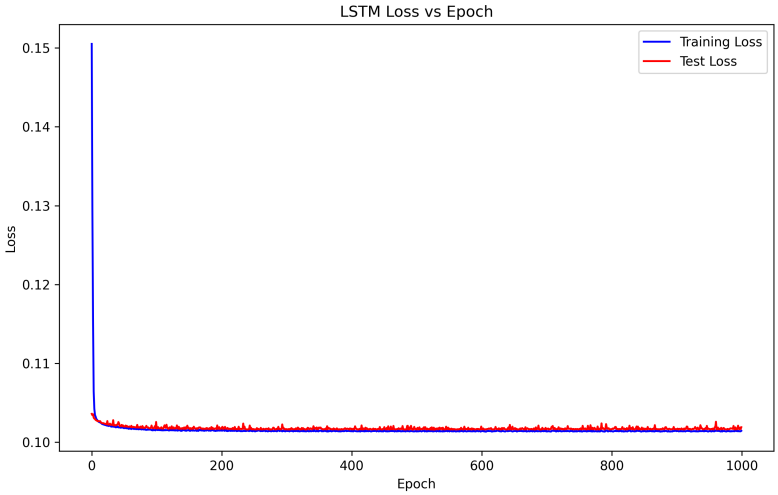


Figure 15: Tuned LSTM Training and Test Loss Curves

We see that both the training and test losses quickly saturate and then fluctuate around a point just above 0.10. This indicates much further training would not have increased the performance of the model, and that once more the hyperparameters chosen ensure that the model is not overfitting to the training data.

6.2.3 Untuned LSTM Loss Curves

Again we plotted the training and test loss curves for the Untuned LSTM model on a linear scale:

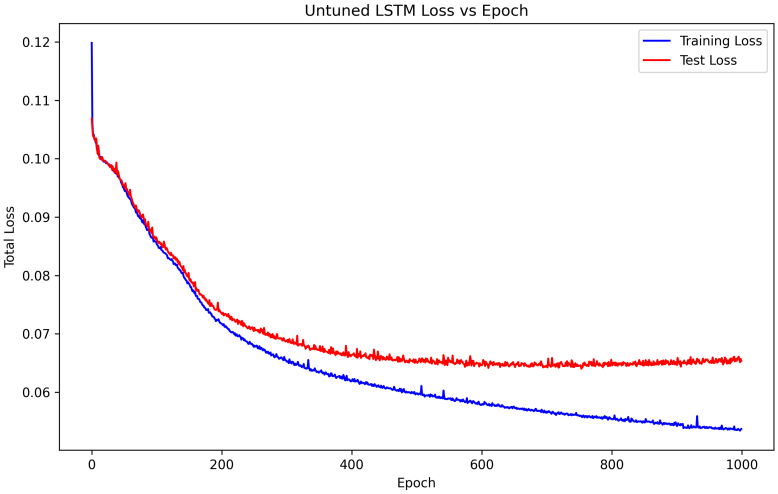


Figure 16: Untuned LSTM Training and Test Loss Curves

This is the most promising of all three loss curves, with the minimum training and test losses achieved being lower than that of the Feed-Forward and Tuned LSTM models. training and test losses decrease in similar fashions. The training loss curve decreases in a much more consistent and linear fashion than the other loss curves, and looks like it would continue to decrease if we continued training, however we note that the test loss begins to increase near the end of the 1000 epochs. This indicates that the model is capable of overfitting to the training data, which makes sense given that the Untuned LSTM had no regularisation applied to it. That being said, the fact that we stopped "early" (before the training loss asymptoted) is in itself a form of regularisation, and the fact that the test loss only just began to increase near the end of the 1000 epochs indicates that this model would still perform well on unseen data.

6.2.4 Hedging Performance (Generated Paths)

We found the PNLs for all models under the 10 000 generated test paths.
The mean and variance of these (PNLs) are compared to those made by a delta-hedging trading strategy informed by Black-Scholes in the table below:

	Black-Scholes	**Feed-Forward NN**	**LSTM NN**	**Untuned LSTM**
Mean(PNLs)	1.3481	0.1780	1.2922	0.0575
Variance(PNLs)	0.4476	4.9558	4.8277	1.7408

Table 7: Comparison of different model's mean and variance of PNLs on Generated Paths

We see that all the neural networks beat Black-Scholes on average, with the feed-forward neural netwok getting a mean PNL an order of magnitude closer to 0, and the Untuned LSTM being two orders of magnitude closer, however for all models Black-Scholes clearly beats them in its variance.

Counter-intuitively, the tuned LSTM has the worse mean of all the trained models, however as mentioned, due to time and computation constraints we were unable to run many trials over many epochs when attempting to find the optimal hyperparameters. Thus, we likely just never found the optimal region during our search. The improved performance of the Untuned LSTM indicates that more complex models might very well lead to better performance, as the Untuned LSTM had no form of regularisation and had two layers as opposed to the single layer found to be optimal during the hyperparameter search. This is however at least promising, as it indicates that even better results are likely achievable with this architecture.

To help visualise the difference in mean and spread of the PNLs, we plotted the PNLs of the Black-Scholes model versus those of the Untuned LSTM model:

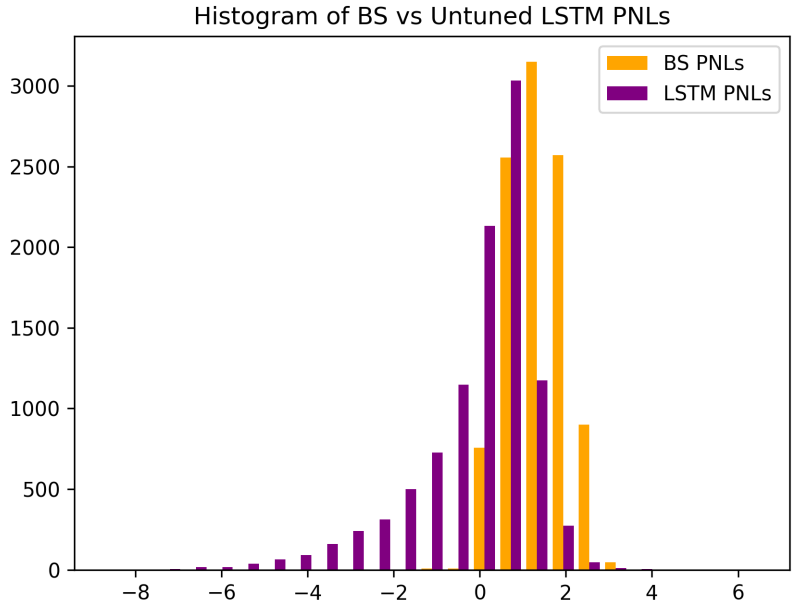


Figure 17: BS vs Untuned LSTM PNLs

6.2.5 Hedging Performance (Final Test)

The final test statistics (as described in the methodology section) for all the models on the real-world 30 day test set are shown in the table below:

	Black-Scholes	**Feed-Forward NN**	**LSTM NN**	**Untuned LSTM**
MSE$(\Delta V_t - \Delta H_t)$	1.8570	0.7452	0.7262	1.3326
Final PNL	-1.3963	-4.8725	-4.4570	0.5634

Table 8: Comparison of model performance for final real-world test set

We see that the feed-forward and Tuned LSTM model portfolios best track the option price on average, however end up having worse PNLs at the time of maturity. This means that the Black-Scholes and Untuned LSTM models made errors earlier on in trading, but then made up for them near the end of training, resulting in better final PNLs. The Untuned LSTM had the best overall PNL, again displaying that it could be used to competitively hedge this option.

It must be noted that the real-world 30 day test path is actually just one sample from the conditional distribution of possible future paths given the past, so these statistics are not truly representative of how these models would perform on average, hence why we additionally did testing on the pseudo-generated paths.

7 Discussion

7.1 Generation

Our findings agree with the overwhelming literature that VAEs can be effectively used as a generative architecture. Our findings also agree with that of the literature Buehler, Horvath, Lyons, Arribas, et al. 2020, Buehler, Horvath, Lyons, Perez Arribas, et al. 2020, Stangroom 2023 in that we can effectively use signatures and VAEs to simulate financial markets in low data environments. Our work also expands upon previous literature in that we simulated the movements of two stocks to effectively generate paths coming from a joint-distribution so that we could hedge a basket option as opposed to a more generic single-stock barrier option.

7.2 Hedging

With regards to the use of Deep Neural Networks (DNNs) to govern hedging strategies, our findings only partially agree with that of the existing literature Stangroom 2023 and Buehler et al. Buehler, Gonon, et al. 2019 in that the trading strategies governed by DNNs can indeed outperform more traditionally informed strategies when hedging a European style option. The findings are only partially agreeable as we found issues of variability in our study which were not found in these other studies.

8 Conclusion

We set out to see whether a DNN could trade assets underlying an Exotic European Basket option so as to replicate the value of the contingent claim, and whether it could do this better than the traditional methods of Black-Scholes.

In order to do this we first attempted to learn the underlying stocks' joint conditional distributions with a VAE. The results of our generation process showed promise in that this approach can effectively learn such distributions, even in relatively low-data environments.

On the actual use of DNN for hedging we had mixed results with our models attaining better mean PNLs on generated paths, but having higher variance than those of the Black-Scholes model. The findings are however promising in that they indicate that with better compute and time, the models could be improved to eventually beat Black-Scholes on both metrics. As such, we conclude that there is clear potential for DNNs to be used to solve the short-comings of the Black-Scholes equations, in that they may more accurately be able to replicate basket price options in incomplete, discrete-time markets.

9 Improvements and Further Research

Given that this was an Honours Project, the scope of this paper is greatly limited. Countless obstacles were met throughout the project and countless mistakes made. The author did their best to remedy these, but it would be against the scientific endeavour not to mention some of the short-comings of this paper. Additionally, the author came across many open questions that they believe would be interesting to explore in greater detail further down the line. This sections aims to address and expand upon both the short-comings and future avenues of exploration.

9.1 Path Signature Inversion

As has been mentioned, the inversion from a path signature back to it's original time-series data is a difficult one. In the other papers which this project mimics (Buehler, Horvath, Lyons, Perez Arribas, et al. 2020, Stangroom 2023), evolutionary algorithms were used in order to find the path inversions. In our case we simply used the Signatory library (Kidger and Lyons 2021). This greatly limited us in a number of regards.

Firstly, we were not able to apply lead-lag transformations to the training data. These transformations help capture the important notion of quadratic variation and co-variation of paths (Gyurkó et al. 2014) and are consequently highly beneficial when attempting to learn and generate financial data. The issue is that the transformation nearly quadruples the dimensionality of the data, meaning that the size of signatures taken blows up exponentially due to the nature of the signature. As a consequence of this, the computational cost associated with inverting a signature of a lead-lagged path is massive, and it meant that we were unable to use this transformation in our implementation.

Secondly, the computational cost of signature inversion limited us to have to learn 5 day paths instead of some greater time window. In theory, larger time windows might better characterise the distribution and variance of the data, leading to better generative capabilities.

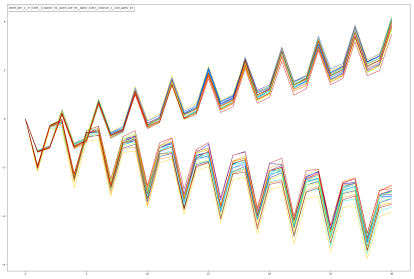
9.1.1 Possible Solution

The author attempted to alter the VAE to take in as input the original path and signature, and simply output the original path instead of outputting the signature. This would completely negate the need to do a signature inversion and hence overcome the issues mentioned. This approach did not work out the gate, with the VAE suffering from latent space collapse and all generated samples appearing identical. Due to time constraints we could not properly investigate if this issue is fixable, but this could offer a potential solution if further investigated.

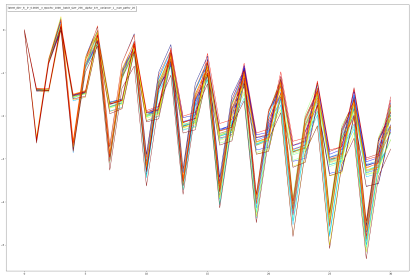
Another possible solution is to train a wholly separate neural network to tackle the task of signature inversion, and add this to the data pipe-line.

9.2 VAE Hyperparameter Tuning

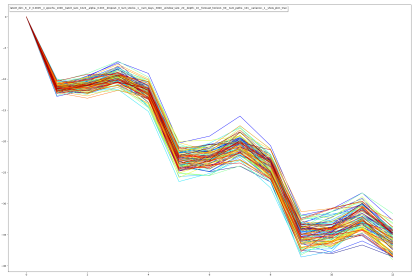
No validation set was used during the training of the VAE, as we did not implement a proper hyperparameter search. For the first two months of work on this project, our VAE failed miserably to generate realistic samples. This was again due to a consistent collapsing of the latent space, meaning that generated samples were nearly identical. A few of the resulting paths of failed training runs are depicted below.



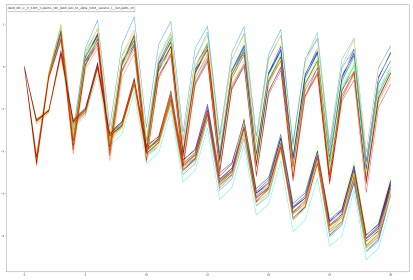
(a)



(b)



(c)



(d)

Figure 18: Failed VAE generations

Once we finally managed to resolve these issues, we no longer had time to perform a proper hyperparameter search and validation analysis.

9.2.1 Solution

The solution to this is of course to perform a proper proper hyperparameter search and validation analysis.

9.3 Trading

In reality performing trades of financial instruments comes at a cost. We did not account for this in our model designs but this would be highly important in reality. This reality is also not captured in the frictionless market assumption of Black-Scholes, so in fact if we were able to effectively account for transaction costs in our models, their performance relative to Black-Scholes would likely be boosted greatly.

9.3.1 Possible Solution

A possible solution to this would be to include a penalty term in the loss functions of the hedging neural networks which penalises the change in holdings from one time-point to another, as this would help limit the number of transactions of the underlying assets made.

9.4 Is Generation Necessary?

As was pointed out by the supervisor of the project, Dr Jonathan Shock, what we are effectively doing in the project is training the neural networks on boot-strapped data, i.e. data that is estimated from the true data. Although the use of generated data gives us a nice proxy metric for the robustness of our models over future possible paths, it would be interesting to see whether or not we could simply use the true data to train a neural network to undertake the task of hedging.

References

Akiba, Takuya et al. (2019). "Optuna: A Next-generation Hyperparameter Optimization Framework". In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

Black, Fischer and Myron Scholes (1973). "The pricing of options and corporate liabilities". In: *Journal of political economy* 81.3, pp. 637–654.

Boedihardjo, H., H. Ni, and Z. Qian (2013). "Uniqueness of signature for simple curves". In: *arXiv preprint arXiv:1304.0755*. Results extended to cover the case of paths with finite p variation, p between 1 and 2, and formulated in terms of log signature. arXiv: 1304.0755 [math.PR]. URL: https://arxiv.org/abs/1304.0755.

Boedihardjo, Horatio et al. (June 2014). "The Signature of a Rough Path: Uniqueness". In: Submitted on 30 Jun 2014 (v1), last revised 15 Oct 2015 (this version, v3). arXiv: 1406.7871 [math.CA]. URL: https://arxiv.org/abs/1406.7871.

Bolen, Tison et al. (2024). *An Investigation of the Kolmogorov-Smirnov Two Sample Test using SAS®*. Presented by Cardinal Health, Advanced Analytics Team, Dublin, Ohio, USA. Accessed: 2024-09-24.

Bossi, Nick (2024). *Hedging Honours Project*. Accessed: 2024-09-23. URL: https://github.com/NickBossi/Hedging-Honours-Project.

Buehler, Hans, Lukas Gonon, et al. (2019). "Deep hedging". In: *Quantitative Finance* 19.8, pp. 1271–1291.

Buehler, Hans, Blanka Horvath, Terry Lyons, Imanol Perez Arribas, et al. (2020). "A data-driven market simulator for small data environments". In: *arXiv preprint arXiv:2006.14498*.

Buehler, Hans, Blanka Horvath, Terry Lyons, Imanol Perez Arribas, et al. (2020). "Generating financial markets with signatures". In: *Available at SSRN 3657366*.

Chevyrev, Ilya and Andrey Kormilitzin (2016). "A primer on the signature method in machine learning". In: *arXiv preprint arXiv:1603.03788*.

contributors, Wikipedia (2024). *Kullback–Leibler divergence*. URL: https://en.wikipedia.org/w/index.php?title=Kullback%E2%80%93Leibler_divergence&oldid=1233784240.

Cybenko, George (1989). "Approximation by superpositions of a sigmoidal function". In: *Mathematics of control, signals and systems* 2.4, pp. 303–314.

Ebobisse, Francois (2024). *The Greates Theorem of FA1: The Hahn-Banach Theorem*. University of Cape Town.

ESPEJO, DANNY (2023). "THE RIESZ-MARKOV-KAKUTANI REPRESENTATION THEOREM". In.

Finance, Yahoo (n.d.). *Microsoft Corporation (MSFT) Stock Historical Prices & Data*. Accessed: 2024-08-17. URL: https://finance.yahoo.com/quote/MSFT/history/.

Graham, Benjamin (2013). "Sparse arrays of signatures for online character recognition". In: *arXiv preprint arXiv:1308.0371*. DOI: 10.48550/arXiv.1308.0371. arXiv: 1308.0371 [cs.CV]. URL: https://arxiv.org/abs/1308.0371.

Graham, Benjamin and Jeremy Reizenstein (2024). *iisignature: Tools for calculating the signature and log signature of a data stream*. https://github.com/bottler/iisignature. Accessed: 2024-08-17.

Gyurkó, Lajos Gergely et al. (July 2014). "Extracting information from the signature of a financial data stream". In: *Oxford-Man Institute of Quantitative Finance, University of Oxford*.

Kidger, Patrick and Terry Lyons (2021). "Signatory: differentiable computations of the signature and logsignature transforms, on both CPU and GPU". In: *International Conference on Learning Representations*. https://github.com/patrick-kidger/signatory.

Kingma, Diederik P. and Max Welling (2019). "An Introduction to Variational Autoencoders". In: *Foundations and Trends in Machine Learning* 12.4. arXiv preprint arXiv:1906.02691, pp. 307–392. URL: https://doi.org/10.48550/arXiv.1906.0269.

Lyons, Terry and Andrew D. McLeod (Jan. 2023). *Signature Methods in Machine Learning*. ORA - Oxford University Research Archive. URL: https://ora.ox.ac.uk.

Mathematics, MIT (2024). *Kolmogorov–Smirnov and Mann–Whitney–Wilcoxon Tests*. https://math.mit.edu/~rmd/edf-ks.pdf. Accessed: 2024-09-24.

Mavuso, M. (2018). *Order out of Chaos 2*. University of Cape Town.

— (2019a). *Mathematical Finance*. University of Cape Town.

— (2019b). *Order out of Chaos 1*. University of Cape Town.

— (2019c). *Stochastic Calculus*. University of Cape Town.

Stangroom, Jake (Feb. 2023). "Deep Hedging in Incomplete Markets". Supervisor: Mr. Melusi Mavuso. Master's Dissertation. University of Cape Town.

Yahoo Finance (n.d.). *Apple Inc. (AAPL) Stock Historical Prices & Data*. Accessed: 2024-08-17. URL: https://finance.yahoo.com/quote/AAPL/history/.

Zhou, S. (2019). "Signatures, Rough Paths and Applications in Machine Learning". Master's thesis. Utrecht University. URL: https://studenttheses.uu.nl/handle/20.500.12932/33648.